Faculty of Engineering Sciences

Heidelberg University

Master's Thesis

in Computer Engineering

submitted by

Bc. VLADISLAV VÁLEK

born in Šumperk, Czech Republic

2026

SHARED CPU-FPGA FILESYSTEM ACCESS ON FAST NVME DEVICES

This Master's Thesis has been carried out by

Bc. Vladislav Válek

at the

Institute of Computer Engineering (ZITI)

under the supervision of

Prof. Dr. Dirk Koch

ABSTRACT

This thesis presents an architecture for shared CPU-FPGA filesystem access to NVMe storage using PCI Express peer-to-peer communication. The goal is to reduce host-memory staging and software overhead by enabling direct accelerator interaction with an NVMe device while preserving standard host-side filesystem usage. The implementation combines hardware and software components: an FPGA IP core (DMA Iuventus) integrated with PCIe endpoint interfaces processes NVMe-related traffic, manages queue interactions, and issues read/write commands; a userspace software stack based on NDK-SW and SPDK, together with Linux ublk, provides control-plane integration and exposes a mountable block device. The complete flow is validated with an exFAT filesystem and demonstrates shared access from both CPU and FPGA contexts. Experimental evaluation on an AMD Alveo U55C platform with an NVMe SSD reports resource utilization and request latency for multiple transfer sizes and access patterns. The results confirm functional operation of the proposed approach and show distinct latency characteristics for read and write paths, with sublinear read-latency growth for larger transfers. The work demonstrates practical feasibility of host-bypass storage acceleration and provides a foundation for future optimization of synchronization, throughput, and multi-device deployment scenarios.

KEYWORDS

PCIe, DMA, host-bypassing, peer-to-peer, SPDK, NVMe, FPGA, userspace, ublk, bdev

Erklärung

Ich versichere, dass ich diese Arbeit selbstständig verfasst habe und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Heidelberg ....

Unterschrift

ACKNOWLEDGEMENT

I would like to thank my supervisor, Prof. Dr. Dirk Koch for his valuable feedback when writing this thesis as well as in my research work at the Heidelberg University. I would also like to thank my colleague, Dr. Riadh Ben Abdelhamid, for providing necessary user feedback for the current course of research. Further, I give many thanks to my colleagues at CESNET, a.l.e., Ing. Jakub Cabal and Ing. Martin Špinler, for mentoring me in the development of PCIe-compliant IPs and software. Finally, I thank my mom, Ing. Marta Válková, for supporting me throughout my studies.

Contents

List of Figures

List of Tables

List of Listings

Introduction

Modern datacenter workloads increasingly combine general-purpose CPUs with accelerators, while also requiring high-throughput and low-latency access to persistent storage. In many practical systems, however, accelerator-to-storage communication still relies on host memory as an intermediate staging area. This approach is functional, but it introduces avoidable data copies when copied from permanent storage, additional software overhead in case the CPU needs to copy the data itselv, and contention on host resources caused by the increased pressure on the memory bus. As storage performance and accelerator parallelism continue to grow, these inefficiencies become a significant system-level bottleneck.

This thesis addresses this problem by proposing and implementing shared CPU-FPGA filesystem access to an NVMe device using PCI Express peer-to-peer communication. The key idea is to let an FPGA accelerator exchange data with the NVMe controller directly, while the host CPU participates only in configuration and standard filesystem operation. To realize this concept, the work combines hardware and software co-design: a custom FPGA IP core handles NVMe-related PCIe traffic and queue maintenance on the accelerator side, and a userspace software stack configures devices and exposes a block-device abstraction in userspace that can be mounted by the operating system.

From the hardware perspective, the thesis begins in chapter 1, where the PCIe standard is introduced as a fundamental basis for the entire work. The chapter also presents the peer-to-peer communication concept at the core of the proposed implementation and explains how entities in the host system are configured and addressed. The standardized PCIe interface used in modern AMD FPGA accelerator cards is then described in chapter 2. The *Integrated Block for PCIe IP* has specific requirements that must be followed when user logic processes PCIe traffic.

The NVMe standard that forms the second key element of the implemented solution is described in chapter 3. The standard has evolved rapidly since its first draft in 2011, and the fundamentals of its functionality are presented here. The command-queue system and controller initialization are explained in detail.

Chapter 4 introduces the *DMA Iuventus* module, integrated with the FPGA PCIe endpoint interfaces. This thesis uses the NDK framework for modular FPGA design and for software access to the hardware. The DMA module processes request and completion traffic, updates NVMe doorbells, serves PCIe reads from accelerator-mapped queue and buffer regions, and submits read/write commands generated for accelerator requests. The design explicitly supports operation with queue and data structures mapped into accelerator BAR regions, enabling host-bypass movement between the FPGA and NVMe device.

The software perspective is described in chapter 5. The thesis uses SPDK with Linux ublk to create a path from userspace control to a mountable block device. This allows the solution to demonstrate shared storage access in a standard Linux environment rather than only raw sector-level tests. The selected *exFAT* filesystem provides a simple and portable target for validating the correctness of read/write operations across CPU and accelerator accesses.

The implemented solution is evaluated in chapter 6 on an *AMD Alveo U55C* accelerator and an NVMe SSD in a custom server platform. Evaluation includes FPGA resource utilization and request-latency measurements for multiple access patterns and transfer sizes. The measurements confirm latency behavior consistent with typical NVMe device operation. The last section of this chapter describes how the provided application is run and how to access the shared filesystem.

Overall, this thesis demonstrates that shared CPU-FPGA filesystem access over NVMe can be achieved without mandatory host-memory data staging by combining PCIe peer-to-peer transport, userspace device control, and an FPGA-side command-processing engine.

1 PCI Express

The *Peripheral Component Interconnect Express* (PCIe) is the interconnect standard that emerged in 2003 from the *Peripheral Component Interconnect* (PCI) standard. The PCIe addressed the drawbacks of the PCI standard, which became increasingly challenging as more devices and more throughput were required. These include:

- Shared-bus topology that, in spite of its simplicity, was rendered unscalable since with an increasing number of devices, the bus gets longer, which weakens the signal for the farthest device.
- Common clock which does not fit well with parallel data transmission model especially when lane skew takes place, meaning the bits of data travel unevenly fast over the wires.
- Half-duplex communication allows only one device to communicate at a time.
- Reflected-wave signaling which was used to reduce the power consumption on the bus. This uses an interference of the sent wave from the master (which had driven the signal to the lower voltage than needed for the digital circuits on the bus) and its reflection that originated on the end of a bus (there is no termination on the bus' end). By the addition of these two waves, the signal reaches the required voltage level for the digital circuits[1].

The PCI protocol did not keep pace with the increasing demand for computer peripheral communication structures and needed to be replaced. The PCI Express protocol significantly changed the strategy for both the communication topology and the physical properties of the interconnect[1].

1.1 Communication model

The basic architecture is shown in fig. 1.1. Connections are implemented as point-to-point links between ports on different device types organized in a tree. The center of the topology is the *Root Complex* (RC), which provides *Root Ports*. Each device communicating over the infrastructure (such as today's GPU/FPGA accelerators) is called *Function* and is the major source of traffic within the system. Each Function contains an *Upstream port* for exchanging data. Since the number of Root Ports is limited by the processor's interconnect, fan-out is provided by *PCIe Switches*, which supply multiple *Downstream ports* and a single Upstream port. Each port supports full-duplex communication and introduces an asynchronous clocking model. Using this model significantly improves the achievable throughput of the interconnect, where the clock gets reconstructed from the patterns in the incoming data on the receiving port. Using Switches, the number of devices can be increased arbitrarily, making PCIe a switched network[2]. Although the PCIe specification introduces

many advanced features, this thesis presents mostly the necessary ones for its purposes.

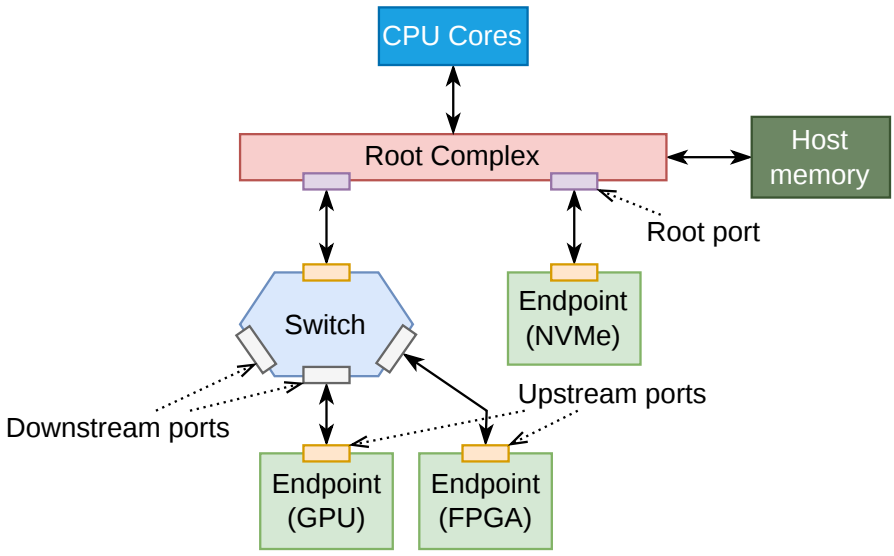

Fig. 1.1: Example of a topology using PCIe interconnect with its designated elements

The PCIe specification defines elementary addressing to identify its devices, composed of *Bus, Device, Function* (BDF) identifiers. Every point-to-point connection in the topology makes a separate *Bus* including internal (virtual) buses of the Switches and the Root Complex. The *Device* identifier remained as a remnant of the PCI standard, and every Bus within the PCIe topology contains only one. Within each Device, one or more *Functions* can be present. The Function is an addressable entity in the configuration space where *Endpoint* is the most important type of a Function. Using the term 'device' in this thesis will refer to a general device in the PCIe topology that can contain one or more Functions (or Endpoints)[2].

Devices communicate using units called *Transaction Layer Packets* (TLPs), forming a packetized transfer. Each TLP contains a PCIe header used for identification followed by an optional payload. There are many TLP types, but the ones used in this work are *Memory Read Request* (MRd) and *Memory Write Request* (MWr). Requests may require a response in the form of *Completion* (Cpl), presenting a third utilized TLP type. Such requests are called *non-posted* (like MRds) while requests that do not require a response are called *posted* (like MWrs). The device creating a request is called *Requester*, while the device 'completing' the request (sending a response or just accepting the payload) is called *Completer*.

RC is located on the CPU die and tightly integrated with the processing cores to provide fast access to the system's I/O devices. RC also provides the connection to host memory, which is the primary communication target used by DMA engines for

Host-to-Device (H2D) and *Device-to-Host* (D2H) transfers. Additionally, multiple Root Ports or Downstream ports enable direct *Device-to-Device* (D2D) communication, as shown in fig. 1.2.

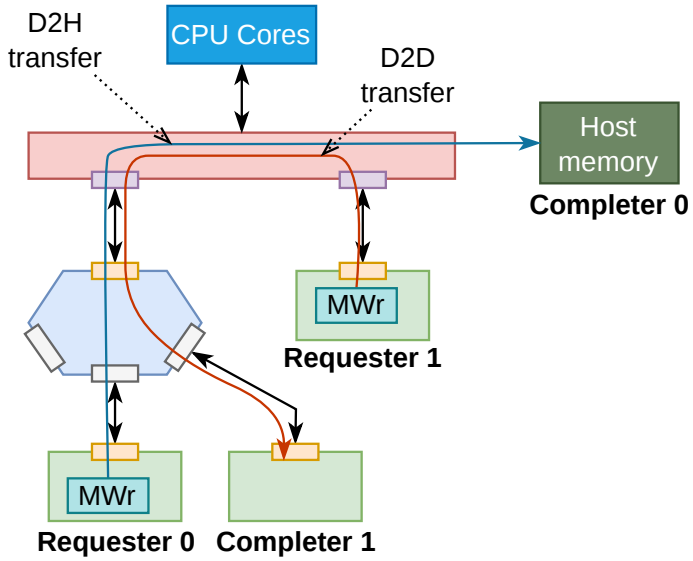

Fig. 1.2: Examples of transports occurring in the PCIe domain. Each device can be both Requester (i.e. dispatching a request) and a Completer (i.e. receiving a request) at the same time.

1.2 Link configuration

The PCIe specification is published in versions called *generations*. With every new specification, the throughput of the bus gets doubled while keeping backward and forward compatibility between its standards (this is an important feature of PCIe). This is the reason why the implementation of the protocol stack remains so complicated, especially on the physical layer. The physical connection between two ports consists of multiple differential pairs called *lanes* that are bundled together into a *link*. Each lane allows for full-duplex communication, where each direction is communicated using a separate differential pair. The link can either contain 1, 2, 4, 8 or 16 lanes. The communication speeds are shown in table 1.1.

As for the physical attachment, this thesis works with two of them, namely *Card Electromechanical* (CEM) and *M.2*. The CEM standard is the oldest one used for *Add-in Cards* with GPU chips, FPGA accelerators or *Network Interface Cards* (NICs). It supports from 1 to 16 lanes where wider link widths support lower lane counts and vice versa (these however with specially carved connectors that fit longer connector of the attached card). This form-factor allows for connection of the

Table 1.1: PCIe throughput based on its versions. The throughput is calculated without the overhead of the Physical layer[3]

Version	**Line code**		**Link throughput [GBps]**				
	Electrical	**Logical**	**x1**	**x2**	**x4**	**x8**	**x16**
Gen3	NRZ	128b/130b	0.985	1.969	3.938	7.877	15.754
Gen4	NRZ	128b/130b	1.969	3.938	7.877	15.754	31.508
Gen5	NRZ	128b/130b	3.938	7.877	15.754	31.508	63.015
Gen6	PAM-4	242B/256B	7.563	15.125	30.25	60.5	121
Gen7	PAM-4	242B/256B	15.125	30.25	60.5	121	242

most power consuming peripherals where the connector can supply up to 75 W[4]. The accelerators can require additional power by introducing external power supply attachments. The most common ATX connector adds another 75 W in the 6-pin or 150 W in the 8-pin configuration[5].

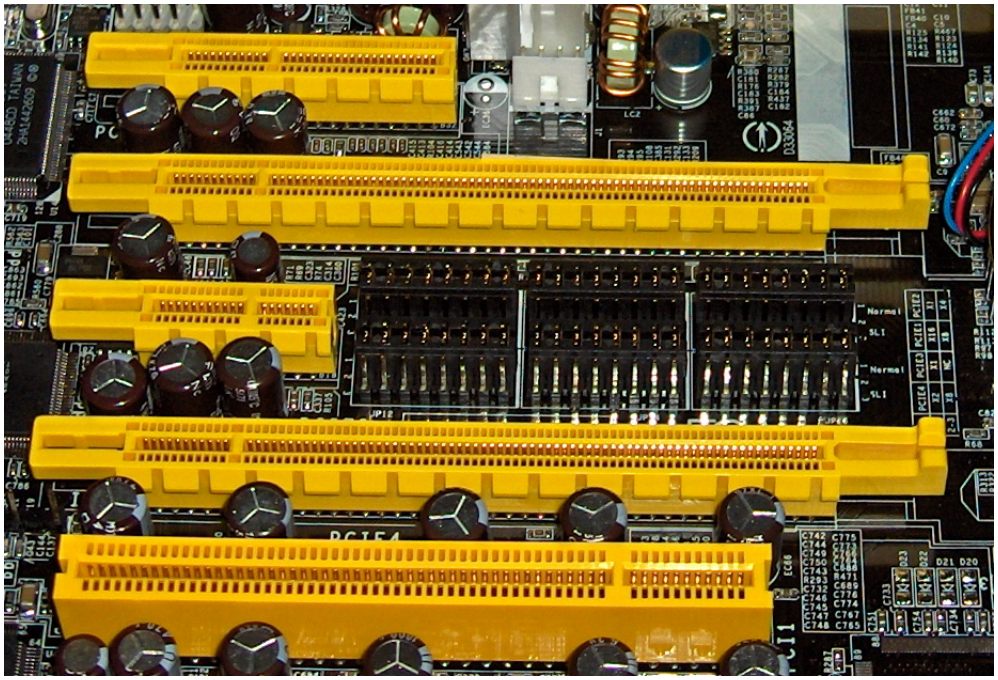

Fig. 1.3: Different types of CEM PCIe connectors supporting 4, 16, 1 or 16 lanes (looking from the top). The bottom connector belongs to the old PCI bus[6]

The second type of physical attachment, *M.2*, is designed for ultra-light, power efficient platforms such as wireless modules or (as in the case of this thesis) *Non-volatile Memory Express* (NVMe) drives. This connector has a fixed set of 4 lanes and a variable module length ranging from 30 mm to 110 mm. These connectors present a detachable form factor for PCIe links but the standard allows for permanently attached devices to be connected as well[4].

As is obvious from the variety of link configurations in terms of data rate and number of lanes, the ports on PCIe devices inside a compute node need to agree on common parameters for every bus in order to facilitate data transfer. This process of negotiation is called *link training* and it takes place after the network gets powered up. The training is done on per-link basis so there are multiple differently configured links in the system at a given point in time. This is presented on fig. 1.4 where the M.2 SSD needs to communicate with the processor (where Root Ports can usually utilize up to 16 lanes) or with a NIC which supports up to 16 lanes. Other than link width and data rate, more parameters are configured such as lane polarity, lane reversal or lane-to-lane deskew. The process of link training happens only in hardware and is transparent for the user[1].

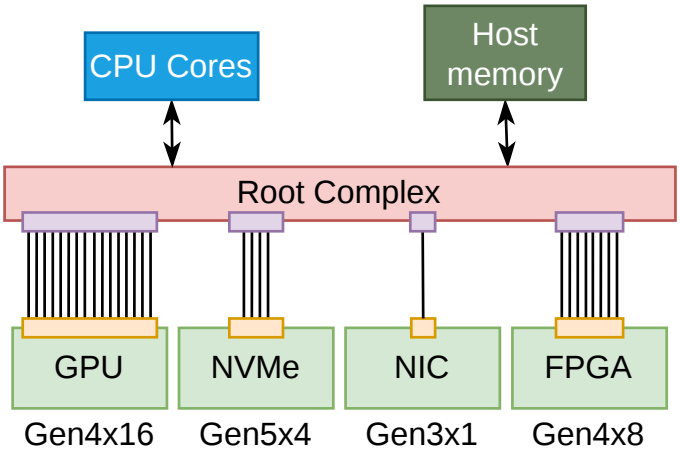

Fig. 1.4: Topology with multiple interconnected devices where each supports different link width.

1.3 Device configuration

After the initial link training is done and every link is active, the host system does device discovery and configuration upon system boot in order to map the network. The process of discovery is called *enumeration* and involves a CPU communicating through the Root Complex in order to find every connected device in the topology and assign its BDF identifier (this includes assigning numbers also to buses between ports that do not necessarily connect to an Endpoint, like two Switches or a Switch with the Root Complex). The search is done in a depth-first way where each device is registered if it responds with a valid *Vendor ID* that is located within device's configuration registers. After assigning all BDFs in the topology, the configuration software further configures registers in the discovered devices. The configuration is

done solely by the Root Complex in order to avoid the complexity of management when multiple devices (e.g. Endpoints) would try to configure each other. The initial assignment of configuration attributes is done using an I/O-based transfer, but further settings are done using *Memory-mapped I/O* (MMIO).

Each Endpoint contains configuration registers in a 64-byte *PCI Configuration Header*, which holds the most important attributes controlling the Function's ability to initiate transactions over PCIe and to be addressable within a host system. Firstly, since devices cannot generate Requests by default, the *Bus Master* attribute is enabled. Secondly, since we want to target the function using memory-mapped transactions (commonly referred to as MMIO)[1]this attribute has to be enabled as well. Thirdly, in order for adjacent devices to address the current Function and to access its internal, user-defined memory space, at least one of the *Base Address Registers* (BARs) must be initialized. By using BARs, each Endpoint requests a memory space within a host system for MMIO access. In case of the attached FPGA accelerator, this is needed to access the internal application logic from the outside since each Endpoint accepts only those memory transactions whose addresses target the ones specified in BARs. Each Function can register up to six 32-bit BARs which provide memory segmentation needed by some host applications. When 64-bit addressing is requested, two consecutive BARs are used[1]. Throughout this work, the observation has been made that most devices allocate one to two BARs and, when not exceeding 4 GiB of space, an operating system maps them to 32-bit address ranges even when 64-bit addressing has been requested. However, this addressing has to be enabled in the UEFI settings of the host system.

Besides the sheer capability for devices to be addressable in the system, other parameters are configured and need to be followed by user applications (e.g. the user-defined FPGA logic). A Function defines its supported *Maximum Payload Size* (MPS) in the *Device Capabilities register*. Seeing this value, the configuration software sets its preferred value in a field in the *Device Control register*. The value of this field prohibits the Transmitter to dispatch TLP with greater payload than MPS specified and the Receiver to process them. The configuration allows further adjustment using the *Maximum Read Request Size* (MRRS) in the *Device Control register* which prohibits the Transmitter to dispatch a Read Request greater than the specified size. Available values for these two parameters range from 128 to 4096 bytes

[1]There are two basic types of transactions based on the mapping they are using. The first are *IO transactions* that were left within the PCIe standard as a legacy mechanism to ensure compatibility with older devices. This mechanism has a limited space for use (4 GiB at most) and even write transactions require a response with a completion status which makes them extremely expensive. The newer software maps the *PCIe Configuration Space* entirely into the memory and ditches IO-based transactions for a *memory-mapped mechanism*. This allows 64-bit addressing that allows to encompass many PCIe Functions in the system[1].

(in powers of two steps). The configuration software balances these parameters based on latency/bandwidth requirements in the network. The configuration has to take into account pairs of adjacent devices communicating with each other, since receiving a TLP whose size is larger than the set MPS on a port results in the function rejecting this packet and reporting a fatal error[2].

1.4 Peer-to-peer transfer

An important aspect of PCIe (with its predecessor as well) is *Direct Memory Access* (DMA) transfer, that allow to copy large amounts of data without the host CPU facilitating such copying. The usual approach is the exchange of data between PCIe devices and the host CPU through buffers in the host memory. However, the network infrastructure allows to address other devices in the topology as well. This type of D2D transport is called *Peer-to-peer* (P2P) or *Host Bypassing* and forms a fundamental part of this thesis. There are several implications of running such transport in the host system. The specification makes the P2P support for each Switch mandatory (for transport between its Downstream ports) but optional for the Root Complex (for transport between its Root Ports) though. The routing of TLP differs based on its type with MWrs and MRds being routed by memory address while Cpl being routed by Requester's BDF identifier[2].

Today's host CPUs utilize *I/O Memory Management Unit* (IOMMU) to translate physical address to virtual ones in order to securely split I/O peripherals into groups[7]. This unit is crucial in multi-tenant environments (like guests as virtual machines) where only devices in one group can perform P2P transfers between each other. Otherwise, where communication between two groups is required, the data need to be analyzed by the CPU. Without IOMMU, a potentially malicious device can write and read from every region in the host system. The unit works hand in hand with the PCIe's *Access Control Services* (ACS) which forces every traffic to pass through the RC even if the devices are connected to the single Switch (this is shown in fig. 1.5). Otherwise, all devices under each Root Port are put to a common IOMMU group which does not play well with the need to decide isolation on per-device basis[8].

Considering the test server used in this work, the support for P2P transfers has been discovered as well as the presence of multiple IOMMUs and enabled ACS on the Root Ports. Since security is not a topic of this thesis nor are the system considerations in the multi-tenant environments, both of these functionalities have

[2]The PCIe standard does not require System Elements to perform segmentation of large TLPs[2].

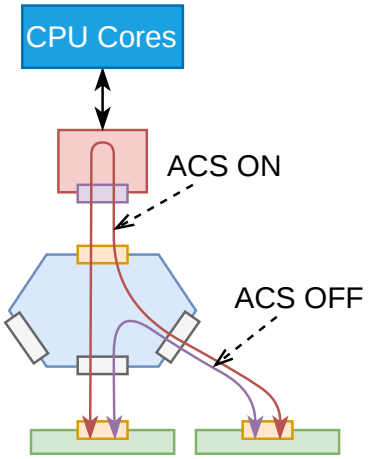

Fig. 1.5: Influence of the ACS mechanism

been disabled in the server's UEFI. Therefore, the PCIe devices communicate using physical addresses.

2 Integrated core for PCIe on FPGAs

Modern FPGAs provide hardened IP cores that implement the whole PCIe protocol stack that can be configured either as a Root Port (not as a RC though) or an Upstream Port (mostly for Endpoint, sometimes for Switch on some devices). The IP utilizes high-speed serial transceivers located on the edge of the chip that connect directly to the CEM-compliant connector on the PCB's edge. This work utilizes the *AMD Alveo U55C* data-center acceleration card which is equipped with `xcu55c-fsvh2892-2L-e` chip (which corresponds to the serial chip `vu57p`)[9]. The available integrated core called *PCIE4C* allows configure the interface to run on Gen3 transfer rate using 16 lanes (Gen3x16) or Gen4 transfer rate with 8 lanes (Gen4x8) providing the same throughput but reducing logic utilization. Since two of these cores are provided close to each other and a total of 16 lanes is provided, these cores can split the available lanes and the acceleration card can be configured to run in a *PCIe bifurcation* mode. This mode utilizes a single PCIe link running two physically separated devices (each containing its own set of Functions). Therefore, the acceleration card can reach a cumulative data transfer rate of PCIe Gen4x16.

2.1 Interfaces to user logic

Since the described core implements the whole PCIe protocol stack (meaning from Physical to Transaction layer), the user logic does not have to deal with the tasks of the lower layers, such as transaction buffering, flow control credit management, data integrity check, lane deskew or clock recovery. In terms of the PCIe topology, this core implements an Upstream port (specifically as a PCIe Endpoint). The core provides four separate interfaces for the user logic where each follows the *AXI4-Stream* specification[10]. As mentioned before, each PCIe port allows for full-duplex communication, allowing each device to act both as a Requester and as a Completer at the same time. The core contains these interfaces exactly in accordance with this behavior, which includes:

***Requester Request* (RQ)** allows user logic to send requests to other components in the PCIe hierarchy.

***Requester Completion* (RC)** transfers completions for non-posted requests dispatched on the RQ interfaces. This interface is not used in this work.

***Completer Request* (CQ)** receives requests from other Functions in the PCIe hierarchy and provides them for the user logic.

***Completer Completion* (CC)** is used by the user logic to send responses for non-posted requests received on the CQ interface.

Other than that, the core provides additional interfaces for transferring status information either from the link or from user logic, dispatching and receiving interrupts, sending messages, etc. The PCIe IP maintains its set of status registers as every other PCIe Endpoint does, and they are accessible by an operating system. A separate interface for the Requester side (i.e. RQ and RC) can be used to manage flow control credits but this is not used in this work and the management is left on the PCIe IP.

The width of the AXI4-Stream interfaces depends on the configured transfer rate of the IP. This work implements the DMA engine using 8-lane Gen4 PCIe endpoint which results in a 512-bit wide bus running on 250 MHz. All of the interfaces are running on the same clock that is provided as the 'user clock' output from the IP[11]. Data units communicated on these buses are TLPs where each is started with a *TLP header* followed by an optional payload of data. The header uniquely identifies a payload within the PCIe topology with a physical address to the reserved space in host memory or in another PCIe BAR. The TLP header has a fixed size of 3 DW (12 B) for RC/CC and 4 DW (16 B) for RQ/CQ interfaces.

Using wide buses carries a significant drawback when data do not fill whole bus words. For example, transferring 1 B of data using a 512b bus utilizes only about 1.5 % of the bandwidth (1 B of payload plus a 16 B TLP header). To mitigate this inefficiency, the bus word is separated into two segments of equal size. Each segment can contain up to one TLP start and/or up to one TLP end. This doubles the throughput for packets that fit entirely within a single segment and generally improves throughput when a packet ends in the first segment while the second packet can then begin in the second segment. This feature is called *Straddling*[11].

3 Non-volatile Memory Express

The NVMe standard emerged in 2011 as a way to integrate non-volatile storage into the PCIe domain[1]. The standard emerged as a successor of *SAS* and *SATA* specifications in order to meet the requirements of consumer systems in terms of latency, throughput and scalability that fit flash-based storage. All of these standards introduced a queue mechanism where commands (like write and read) are present and processed by the controller of the attached *Non-volatile Media* (NVM). Both SATA and SAS support one command queue allowing up to 32 and 256 commands respectively. NVMe enhances this mechanism by allowing to instantiate up to 2^{16} queues where each can contain up to 2^{16} commands[12]. Having many queues results in hardware-aided parallel access to the NVM where, for example, each CPU core receives its own queue. The schematic of the NVMe device is depicted in fig. 3.1. The *NVMe controller* sits on top of a PCIe controller, managing the instantiation of queues and processing of commands. The non-volatile storage is presented as a *Namespace* which is the basic referenceable unit of storage in the NVMe standard[2]. A namespace is split into multiple sectors of *Logical Block Addresss* (LBAs) which are at least 512 B in size. The controller is equipped with a DMA engine that performs a read and write of raw data through PCIe between memory regions in the host system. This implies that the NVMe device behaves as a Bus Master and can therefore generate its own MWrs and MRds which is suitable for the P2P communication in the PCIe domain.

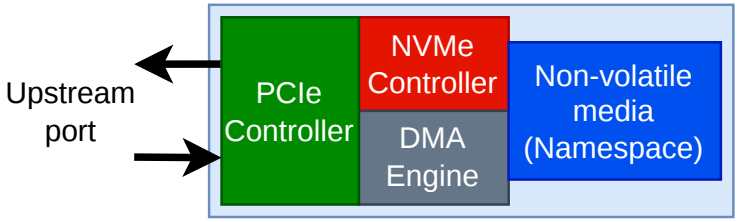

Fig. 3.1: Elementary block scheme of the NVMe device

The command mechanism provides two types of queues that always occur in pairs, the *Submission queue* and the *Completion queue*. A Submission queue contains commands that need to be processed by the NVMe controller which, after a

[2]This thesis works with specification version 1.2b since it describes only its basic concepts and not every currently working NVMe device supports the newest standard. The NVMe specification is designed to be backward compatible though. Using older specifications may be advised for onboarding users to quickly understand basic concepts.

[2]This is a simplified view in the referenced version of the NVMe specification. Later versions define a more complex and fine-grained classification beyond namespaces. Moreover, there can be many namespaces maintained by a single NVMe controller as well as one namespace shared by many controllers.

command's execution, returns the result of the operation into a Completion queue. These queues are allocated as circular buffers in host-addressable memory. The most important pair is the *Admin queue-pair* that is used for the startup configuration. The controller appears in the PCIe system as a single Function and, initially, the kernel is only able to reach the standard registers in the PCIe Configuration Space and its BARs. The NVMe driver allocates Admin queue-pair and publishes base addresses and sizes of the queues to controller registers located in BAR0.

For data transmission to/from the NVMe namespace, the configuration driver instantiates one or more *I/O queue-pairs* which are used by the consumers/producers of data. Same as for the Admin queue-pair, these queues are allocated circular buffers in host-addressable memory. A fully initialized system with an 8-core processor and an NVMe device is shown in fig. 3.2. The controller can associate multiple I/O Submission queues with one I/O Completion queue in order to save resources (this is not possible for the Admin queue-pair though since only one can be instantiated). Each queue pair is identified by its ID where Admin queue-pair has a fixed ID of 0 and every other I/O queue pair reserves an ID of a higher index.

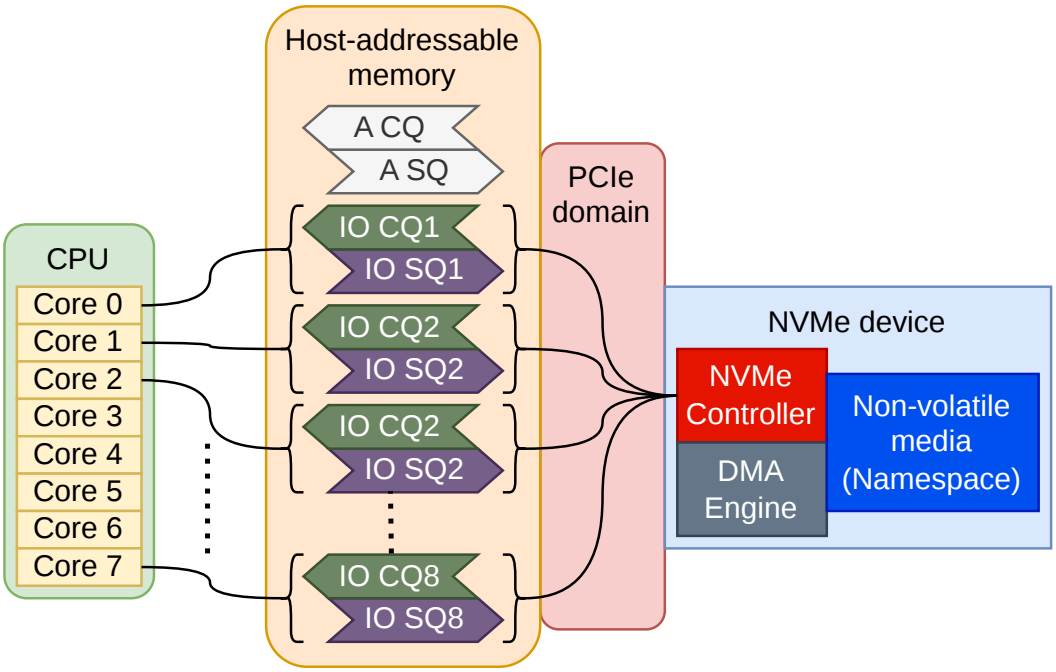

Fig. 3.2: Initialized system with an NVMe device communicating with the host CPU. The indexing of the IO adheres to the standard since the index 0 is reserved for the Admin queue-pair.

3.1 Principles of operation

One queue requires two pointers for its operation, a *Head doorbell* and a *Tail doorbell*. A consumer of entries from the queue operates a Head doorbell whereas a producer operates a Tail doorbell. This pair of pointers ensures that a producer does not overwrite unprocessed entries and that a consumer reads only valid ones. A queue is considered empty if values of these pointers are equal, and full if the value of the Tail doorbell is equal to the value of the Head doorbell minus one modulo the size of the queue as shown in figs. 3.3 and 3.4. The full queue condition implies that the effective size of a queue, i.e. the maximum amount of entries that can reside in the queue, is the size of the queue minus one.

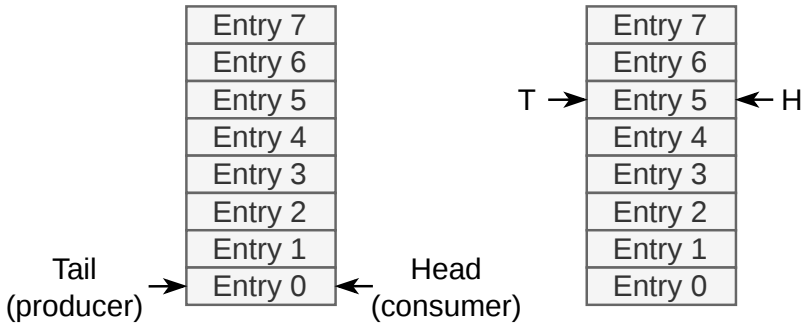

Fig. 3.3: Two examples of an empty queue

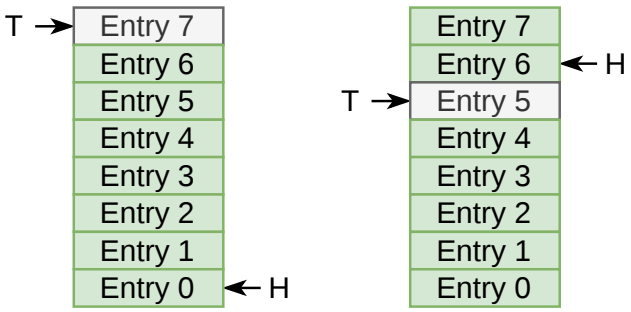

Fig. 3.4: Two examples of a full queue

A master submits a command to a submission queue by using a template for *Submission Queue Entry* (SQE) described further. Upon storing such entry in the queue, the master updates the *Submission Queue Tail Doorbell* (SQTDBL) pointer in controller's registers (in BAR0) in order to signalize to the controller that new command has been submitted. This is followed by the controller fetching and executing the command that is finished by dispatching a completion into a completion queue that uses the *Completion Queue Entry* (CQE) template. If a command operates with additional data (like reading/writing data from/to the NVM or fetching

commands from a submission queue), the controller engages the DMA engine on the device. To detect a received completion of a command, the master does not have to read the *Completion Queue Tail Doorbell* (CQTDBL) value (since this is not available and remains internal to the NVMe controller anyways). Rather, every CQE contains a *Phase tag (P)* that indicates that a position in the Completion queue contains a valid entry.

3.1.1 Command submission

The SQE is created by a master by following the format in fig. 3.5 which uniquely defines a command within a single NVMe controller. The entry has a length of 64 B (16 DW) with necessary information for a controller to process a command. There are two sets of commands defined by the NVMe specification, namely *Admin Command Set* and *NVM Command Set*[3]. The most frequently used runtime commands in this thesis are read and write from/to the NVM.

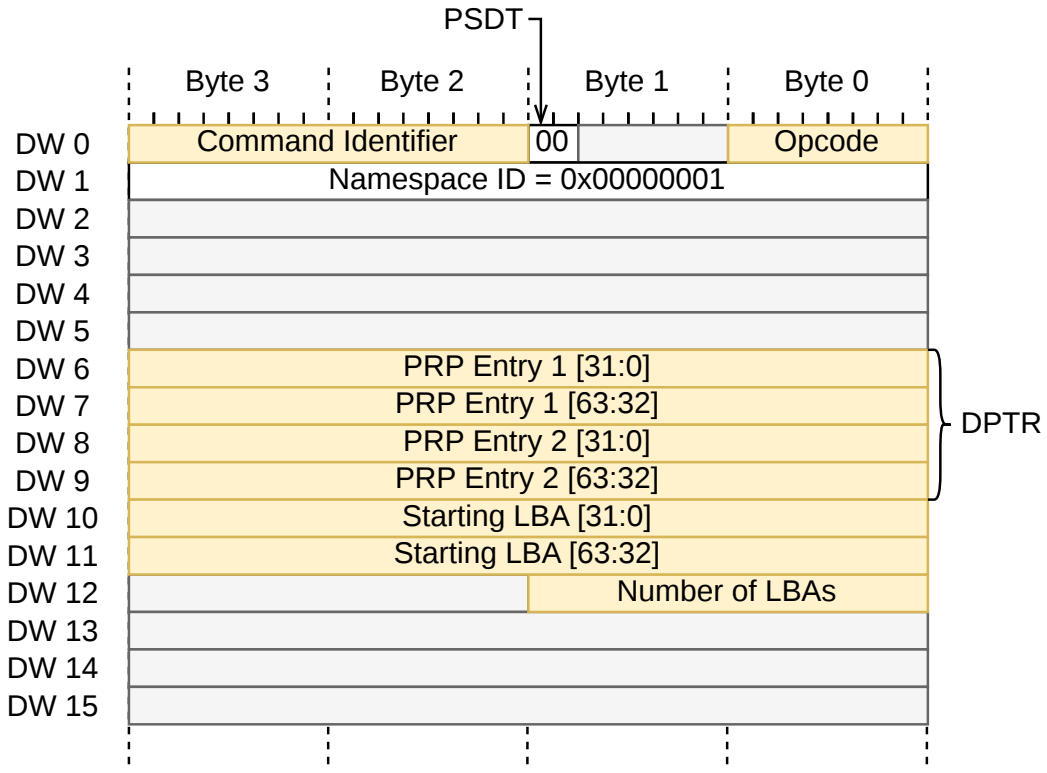

Fig. 3.5: Format of Submission Queue Entry. The greyed-out fields are either reserved or unused within this thesis.

[3]In terms of more recent specification revisions, these sets use the *memory-based* transport which differs from the *message-based* transport. While the first type of transport is typical for PCIe, newer specifications allow to attach NVM devices to other fabrics, like RDMA or Ethernet, that use the second type of transport. These are part of the *NVMe over Fabric* (NVMe-oF) standard published separately.

An SQE begins with the *Opcode* (OPC) field that distinguishes between different types of commands, the *PSDT* field that specifies that *Physical Resource Pages* (PRPs) are going to be used as copy buffers and the *Command Identifier* (CID) field that provides a unique identifier when combined with the Submission Queue ID. This combination serves as a tag to match the dispatched command with a received completion. The first command DWord is followed by *Namespace Identifier* (NSID) to index a specific namespace that is managed by the adjacent NVMe controller (the index is 1-based, meaning that the first namespace receives NSID 1). The following *Data Pointer* (DPTR) field serves as a pointer to the host-addressable memory either from which data need to be fetched (as for the write command) or to which the data are copied from the NVM (as for the read command). The data pointer is split into two PRP entries depending on how many memory pages span the copied data in host memory, thus how big the transfer is going to be. If the transfer fits into one page, only *PRP Entry 1* (PRP1) is used and it points to the start of the user data. In case a transfer spans over 2 memory pages, the *PRP Entry 2* (PRP2) is used as well to point to the second page. If a transfer spans more than 2 pages, the PRP2 changes its meaning to point to the *PRP List* which contains pointers to every other page that contains user data (meaning from the second one onwards). The following two Dwords (DW10 and DW11) contain the *Starting LBA* (SLBA) field pointing to the namespace followed by the last usable field in DW12, which contains the *Number of Logical Blocks* (NLB) to copy. The number in this field determines the size of a transfer for a currently processed command. The size is limited by the NVMe controller and every master has to consult controller parameters in order not to submit larger transfers. When larger data need to be copied, the master has to split the operation into multiple I/O commands.

3.1.2 Command completion

After a command is processed, the controller dispatches a CQE to the Completion queue to notify the master about the status of a command. The template of this entry is shown in fig. 3.6. CQE is at least 16 bytes in size. The first DWord is used by some commands to publish command-specific information (the Read and Write commands do not use them though) while the second DWord is reserved. The NVMe controller indicates the internal value of *Submission Queue Head Doorbell* (SQHDBL) through every completion entry which notifies the transmitter of commands about how many commands have been read by the controller (thus providing a backpressure). The *SQ Identifier* (SQID) and CID fields serve as a unique identifier of a command for the used NVMe controller. The SQID is present because the NVMe specification allows associating multiple Submission queues with

one Completion queue (this applies for one NVMe controller though since every controller maintains its own set of queues). A *Phase Tag (P)* field follows and its value indicates that the current position in the Completion queue is valid (meaning the position to which *Completion Queue Head Doorbell* (CQHDBL) points to). The value that is considered valid is flipped upon every CQTDBL rollover, starting as logical 1 for the initial pass through queue (i.e. after queue initialization), then flipping to 0 when the pointer rolls back to 0, then to 1 again and so on.

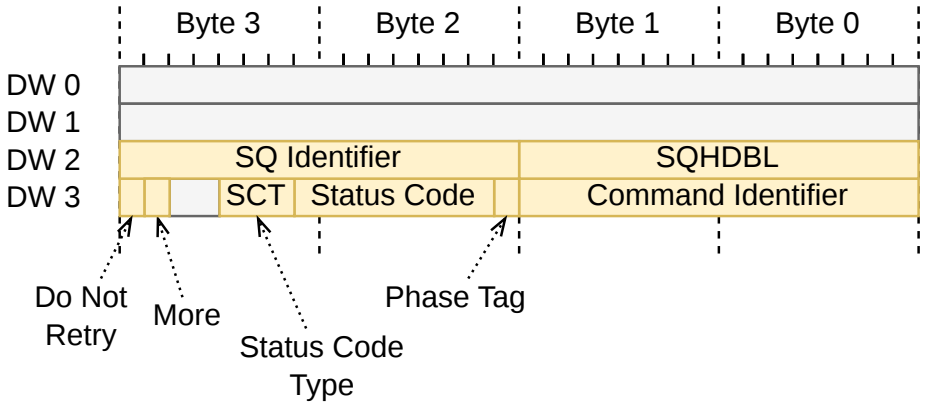

Fig. 3.6: Format of Completion Queue Entry. The greyed-out fields are either reserved or unused within this thesis.

The last bits of a CQE are occupied by status information about the successful/unsuccessful execution of a command. The *Do Not Retry* (DNR) bit indicates that if the same command is going to be resubmitted, it is expected to fail. The submitter has to wait until this bit clears to 0 in order to receive valid data. The *More (M)* bit indicates that a completion for a command is going to be composed of multiple CQEs. More specific information about a command's status can be found in the *Status Code Type* (SCT) and *Status Code* (SC) fields. The NVMe controller provides a coarse-grained category of status within the first field, whereas a more fine-grained one in the second. Here are some examples of possible status codes received:

- A successful completion is indicated by the SCT field set to 0h (i.e. *Generic Command Status*) and the SC field set to 00h (i.e. *Successful Completion*).
- When a submitter dispatches an NVMe command with a specified LBA range that exceeds the capacity of the Namespace, the SCT field is set to 00h and the SC field to 80h (i.e. *LBA Out of Range*).
- Submitting two NVMe Write/Read commands with the same CID results in a completion status where SCT is set to 1h (i.e. *Command Specific Status*) and SC set to 80h (i.e. *Conflicting Attributes*).

3.1.3 Ordering

An important note has to be made with regard to the ordering of submitted commands. An NVMe controller arbitrates between its own set of submission queues in a round-robin fashion and can fetch multiple commands out of every queue[4]. The order of execution where many commands are fetched is not defined by the NVMe specification and can be arbitrary. This is the reason why a tagging of commands using the CID field is important so the submitter can bind a completion to a dispatched command. The order of execution can therefore be derived from the order in which the completions for multiple commands have been received. If strict ordering of commands is required, the submitter needs to implement its own mechanism for that. However, by the means of the NVMe specification, there are two ways of how to ensure ordering. The standard allows to use *Fused operations* in order to force ordering of two subsequent commands. Such two commands can be executed atomically, resulting in the success only if both of the commands succeed. However, this feature is optional and not every device contains it. The second way that is always available, is to submit only one command to the submission queue and wait for its completion before submitting another command. This can be guarded by the submitter itself or by initializing the submission queue with the size of 2 (since this can contain at most 1 valid SQE) and using the blocking by the doorbell pointers. This way is used in the implementation described further in this thesis.

3.2 Multi-pathing

Since multiple queues can be initialized for a single NVMe controller, multiple submitters can execute commands on the controller. Since the proposed implementation in this thesis introduces a direct NVMe device access from an FPGA acceleration card using the PCIe P2P transfer, it contains a potential to access multiple NVMe devices (the 1-to-M topology) as well as to access multiple NVMe devices from multiple acceleration cards, thus implementing an N-to-M topology[5] as seen in fig. 3.7. This feature that allows to access one NVMe device from multiple masters is sometimes called *multi-pathing*.

[4] Advanced devices can implement a Weighted Round-robin or some vendor-specific mechanism.

[5] This topology, however, is only logical since it is made only by addressing and not by physical connections. When performance (like throughput or latency) is critical, it should be adapted on the underlying PCIe topology which is organized as a tree by design.

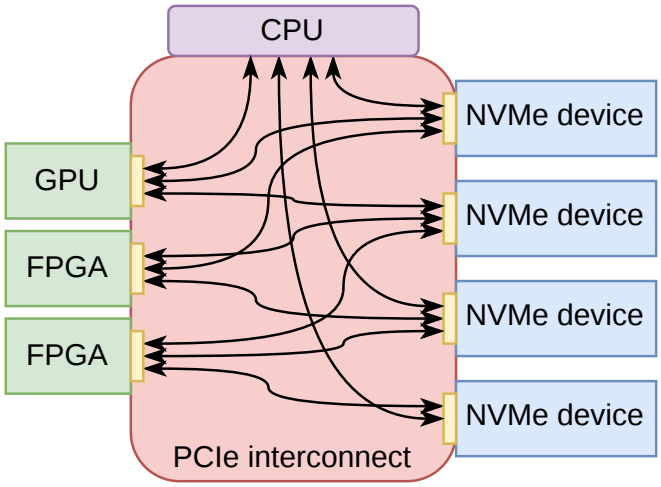

Fig. 3.7: Example topology with N:M mesh network

4 DMA Iuventus

Since this thesis proposes a solution that can access the filesystem located on an NVMe device shared between the host OS and an FPGA accelerator, a specific IP core is needed on the accelerator's side. Without this, the user logic inside an FPGA would deal with raw PCIe data not only from the adjacent NVMe device but from the host system as well. Therefore, the *DMA Iuventus* IP core has been developed to process requests between an accelerator and an NVMe device[13]. The term "DMA" has been retained to highlight the mechanism of the direct copying of data to the adjacent storage without a host CPU facilitating such transfers (this transfer in a P2P manner is sometimes called *Host bypassing*). This IP sits in the FPGA on the integrated block for PCIe to accept NVMe-specific traffic alongside logic that processes transactions to *Configuration and Status* (C/S) registers[1].

4.1 TLP Headers

The DMA IP utilizes 3 of the 4 interfaces provided by the integrated block for PCI Express, namely RQ, CQ, CC. Each one of these uses its own template of the TLP header that the DMA core needs to create or process. The RQ header has a size of 4 DW and its format is depicted in fig. 4.1. The *Address* field is aligned with respect to DWords since its two bottom bits are reserved[2]. For MWrs, the *Dword count* field specifies the size of the payload following the header, whereas, for MRds, it specifies the required amount of data that a Completer needs to send in its Completion. The type of a request is defined in the *Req type* field. The *Func* field informs the PCIe IP which Function generated the request (more on that later)[11]. The last used field is the *NoSnoop* bit, which instructs the host CPU that the underlying address space targeted by the transaction does not require hardware-enforced cache coherency[2]. This means that the address space is not shared by any other device in the system. The RQ interface is used to dispatch SQTDBL and CQHDBL updates to the NVMe registers.

The CQ header (Figure 4.2) follows a format similar to the RQ header, where the user logic acts as a PCIe Completer. The *Target function* field indicates to which Function a request is targeted. When multiple BARs are used, the core puts the index of the specific one in the *BAR ID* field. The size of a targeted BAR is determined by the *BAR Aperture* which serves as a mask for the address (for example, the value of 12 signifies that the BAR has a size of 4 KiB and, therefore,

[1]The source files of the module are available in the cited repository under the `comp/dma/dma_iuventus` directory

[2]The x86_64 architecture addresses its physical memory space with resolution to bytes.

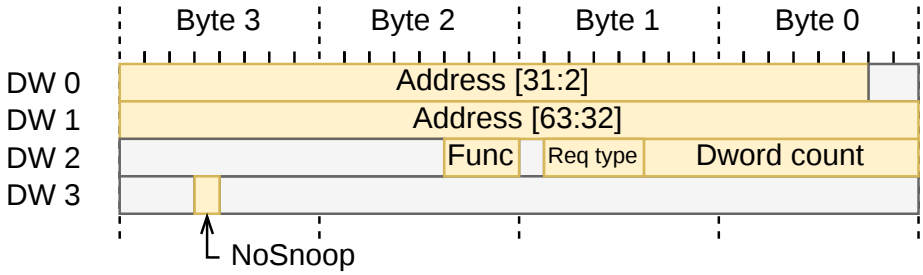

Fig. 4.1: Format of the header used on the Requester Request interface

the address bits [63:12] can be ignored)[11]. Other fields of this header are either reserved or used to create a response for MRd.

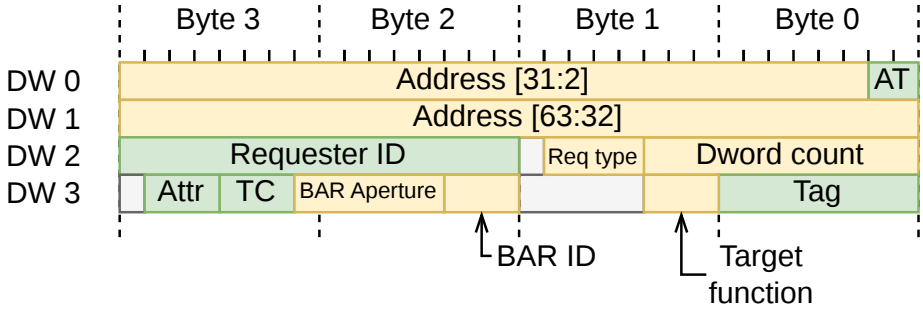

Fig. 4.2: Format of the header used on the Completer Completion interface. The yellow fields are used by the DMA Iuventus core itself whereas the green ones are only used to create the CC header of the response.

For posted requests, such as MRd, the response is dispatched on the CC interface by using the 3 DW TLP header in fig. 4.3. The header may be followed by a payload. The *Address* is a byte-level address of the first byte of the memory block being transferred. The size of the payload is specified in the *Dword count* field. Because the CQ header contains only DWord-aligned address, the DMA logic needs to calculate the byte-level offset by reading the *TUSER*[10] signal of the AXI-Stream bus on the CQ interface which contains the *First Byte Enable* (FBE) as a mask of valid bytes in the first DWord of the payload. The byte offset is then adjusted by factoring initial valid/invalid bytes in the calculation. There is another field used, namely the *Last Byte Enable* (LBE) which is located in the same AXI-Stream signal. This mask serves for the last DWord of the payload to distinguish valid bytes. An exact byte length of the payload can be calculated by using the Dword count, FBE and LBE fields[11].

Each completion transfers a status of either *Successful Completion*, *Unsupported Request* or *Completer Abort* in the *Status* field. Similar to the Target Function field in the RQ header, there is an index of a completing Function. Other than that,

there are multiple fields that need to be copied from the Requester TLP header, such as *Address Translation* (AT), *Requester ID*, *Tag*, *Transaction Class* (TC) and *Attributes* so that the Completion is correctly routed to the Requester and then correctly classified within its internal system. This applies especially for the Tag field which marks all Completions for a specific Request. A Completion can be dispatched in multiple TLPs (especially if the MRRS of the Completer is bigger than the MPS value), which is controlled using the *Byte count* field. The value indicates the remaining amount of bytes that are needed to complete a request including the length of the current payload. The last Completion TLP has the value of *Byte count* equal to the byte-length of its payload[11].

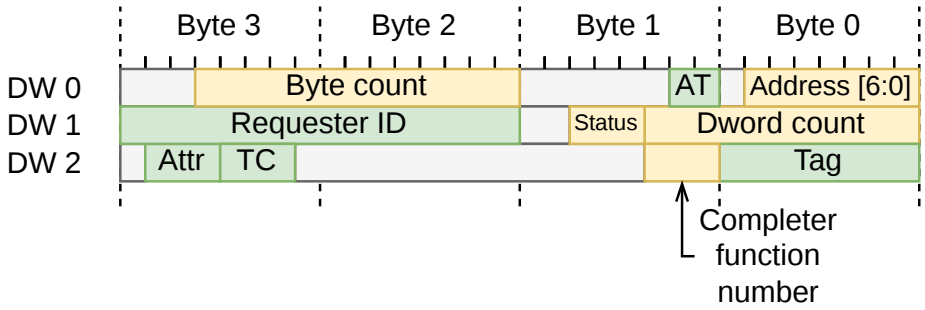

Fig. 4.3: Format of the header used on the Completer Completion interface. The yellow fields are generated by the DMA Iuventus core itself whereas the green ones are copied from the CQ header.

4.2 Software configuration

This thesis uses the *NDK-FPGA* framework developed by *CESNET, a.l.e.*[14]. This framework serves as a development platform for datacenter-grade FPGA acceleration cards providing stable communication infrastructure and allowing to instantiate custom user logic. In order to provide the software control, the *NDK-SW* package is delivered along with the previous framework[15]. This package allows access to internal C/S registers of various components within the FPGA logic from userspace. Additionally, this thesis utilizes the *Storage Performance Development Kit* (SPDK) framework in order to correctly configure an NVMe device and establish a P2P communication chain between an accelerator card and the storage drive. SPDK is a userspace framework to create high-performance applications where drivers are running in the userspace[16]. This enables fast prototyping without the need to operate with kernel modules which can cause fatal system failures if not done carefully[17].

A device that is normally maintained by a kernel driver can only be accessed by a defined API that usually utilizes the *sysfs* filesystem or, when used in custom

applications, the *ioctl* calls with defined flags[18]. Upon every modification of such driver, it needs to be compiled and inserted into the kernel, not to mention the limited debugging options. This mechanism, however, prevents malicious applications from accessing the MMIO resources directly. In order to move all of the control to the userspace, a device needs to be unbound from a particular driver and bound to a new driver[3]. The Linux kernel stack provides two "dummy" drivers that serve the purpose of this thesis. If IOMMU is disabled, the system operates only with physical addresses, the device needs to be bound to the `uio_pci_generic` driver, whereas, if enabled, the system operates with *I/O Virtual Addresss* (IOVAs) and the `vfio-pci` driver needs to be used. Both of these drivers expose devices' BARs to userspace where they can be managed using MMIO. Since the first approach uses disabled IOMMU, the application needs to be run as the root user[17, 19, 20, 21].

4.2.1 Memory partitioning

The fig. 3.2 presents a standard customer system with a main NVMe drive and an instantiated set of queues for every core on the processor. The queues that are created for this purpose are normally stored in the host memory where the 2 buffers get allocated for every queue pair. The data buffers are dynamically generated since values of PRP1 and PRP2 can vary with every dispatched command, therefore conforming to the very principle of the DMA where data do not have to be copied to special intermediate buffers but the pointers can directly reference user data in the host memory.

Since this work wants to establish the P2P communication chain between an FPGA accelerator and an NVMe drive which bypasses the host system, the mapping of queues as well as copy buffers has been moved entirely to the PCIe BARs of the accelerator. Such system is depicted in fig. 4.4. Because the mapping of PCIe BARs of the accelerator is constant, the PRP1 and PRP2 fields are always pointing either to the *Read Buffer* (which contains data that the NVMe needs to fetch as a part of the write command) or to the *Write Buffer* (which is a reserved space where the NVMe uploads its data as a part of the read command). These buffers are allocated in BARs 3 and 2 respectively. The queue pair is located in the first two BARs where Submission Queue is located on BAR 0 and Completion Queue on BAR 1. By mapping all of these data structures to the PCIe device, the copying of user data through host memory is entirely removed.

To configure this P2P chain through SPDK, both the NVMe drive and the accelerator must be unbound from their default kernel drivers.

[3]For concrete steps of how to unbind a device, check the chapter A

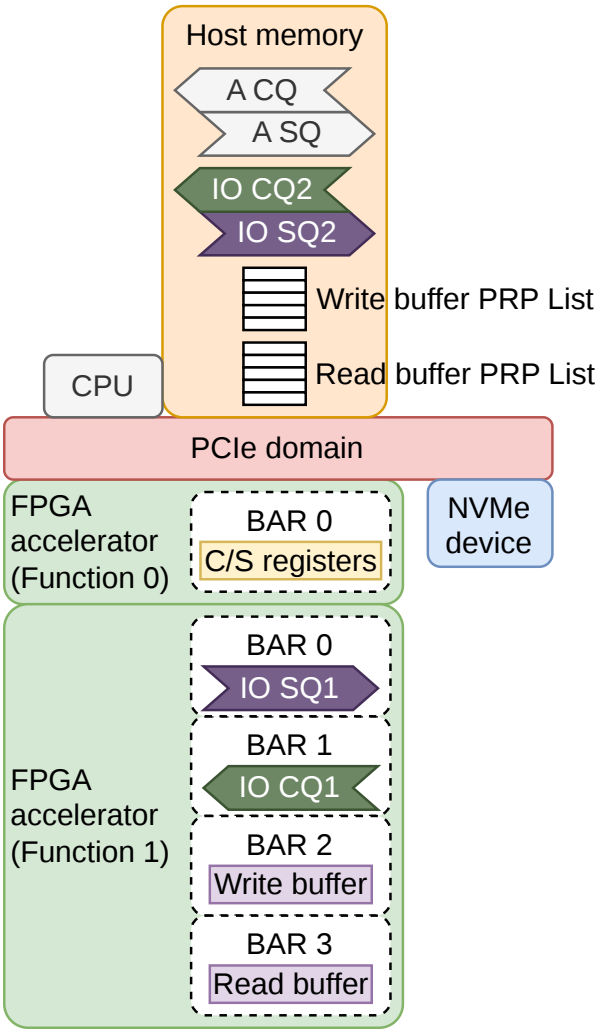

Fig. 4.4: Overview of allocated memory resources as initialized by the SPDK application.

The *NDK-SW* framework provides its own kernel driver in order to operate on the PCIe device and to access its C/S registers. The NVMe device can be entirely unbound since no original kernel driver is used and everything considering a device initialization is handled by the SPDK framework. On the other hand, a different approach had to be chosen for the accelerator, namely to create a second PCIe Function on the device. This allows two drivers in the host system to operate on one device while keeping separate Configuration Space for each one of them. Having this, the access to the configuration registers of the FPGA's user logic can be preserved while a custom driver for the P2P communication chain can be used besides that. Since there is only one PCIe IP per one physical connector on the FPGA chip, all of the requests pass through the same interfaces to the user logic and have to be split according to the function index and the BAR ID[11].

4.2.2 Queue initialization

This step relies on the initial configuration of the controller done by the SPDK framework. Especially, this involves the allocation and assignment of Admin queue-pair whose addresses are stored in the controller's registers (remember that these registers reside in the PCIe BAR of the NVMe drive). The NVMe controller has two ways of configuration, either through its registers (works every time) or through its Admin queue pair (after it gets allocated) using the *Set Features* command. The following list describes the specific case for the solution presented in this thesis and not the general one presented in the specification.

1. The *Controller Capabilities* (CAP) register indicates the maximum supported size of its queues and the requirement if queues need to be physically contiguous.
2. The host allocates space for the Admin Completion and Submission queues with its preferred size and writes these to the *Admin Queue Attributes* (AQA) register. Their base addresses are then written to the *Admin Completion Queue Base Address* (ACQ) and *Admin Submission Queue Base Address* (ASQ) registers. The Admin queue pair has an implicitly reserved Queue ID 0.
3. The host submits a *Set Features* command with the *Number of Queues* attribute set to the desired amount of I/O Submission and I/O Completion queues. The controller responds with the amount of queues that it supports which can be equal or lower than the number of the queues requested.
4. The Queue ID 1 is reserved for the queue pair residing in the FPGA accelerator. This reservation is not only internal to the SPDK framework but also necessary for correct registration of queues on the NVMe controller to keep consistent state in the whole stack when more queues need to be added[4].
5. The host allocates I/O Completion queue in the BAR 1 of the FPGA accelerator and clears its content. The clearing is necessary since the DMA Iuventus logic could otherwise process past entries that have a *Phase Tag* field set to 1 (which is the initial value that is considered valid after the queue initialization).
6. The *Create I/O Completion Queue* command is submitted in order to register the queue on the controller.
7. The host allocates an I/O Submission queue in the BAR 0 of the FPGA accelerator and submits the *Create I/O Submission Queue* command to register this queue on the controller.
8. The Queue ID 2 is reserved for the queue pair for the host to create and maintain a filesystem on the NVMe storage.
9. The host allocates regions in the host memory that are going to hold its I/O queue pair. The Completion queue is cleared in the same way as in the case

of the FPGA accelerator.

10. The queues are registered in the same order as previously, first the Completion queue, then the Submission queue.
11. The system is now ready to operate and write/read raw data to/from the non-volatile storage.

The current configuration can be read by three means similar to the manners in which a startup attributes are written. The BAR 0 of the NVMe controller can be consulted or the *Get Features* command can be submitted to the admin queue-pair. Moreover, the NVMe controller can provide more detailed information by submitting the *Identify* command. This command provides a substantial amount of information with regard to the capabilities of either the controller or a namespace attached to it. This information is then copied to the *Identify Controller* and *Identify Namespace* data structures in the host memory[22].

4.3 Command submission

The P2P chain between the FPGA accelerator and the NVMe drive has been partially established by creating the accelerator-owned queue pair. In order to perform the full P2P communication, the host allocates the BAR 2 and BAR 3 of the accelerator and publishes its physical addresses to the DMA Iuventus module so the submitted commands to the NVMe target the internal buffers of the FPGA.

The *PRP List* is created in the host memory as part of FPGA device initialization as well. This list resides in the host memory for every requester regardless if the accelerator or the host engages in the communication. The values of its elements for the FPGA accelerator are constant (that is because the mapping of BARs does not change) whereas, for the host side, it is created on the go[5]. Although the PRP List can be arbitrarily long and thus support arbitrarily long transfers, the NVMe controller limits this. This limit is published in the *Identify Controller* data structure in the *Maximum Data Transfer Size* (MDTS) field. This applies to the write command as well.

4.3.1 Read command

The read command to the NVMe consists of several steps as seen in fig. 4.5. The command is submitted by writing one SQE data structure to the Submission queue

[4]Remember that the Queue ID is used in the CQE as a part of a command's unique identifier since multiple I/O Submission queues can share one I/O Completion queue

[5]The API of the SPDK framework already contains built-in functions to dispatch read/write commands to the NVMe device and the maintenance of the PRP List is, therefore, left to its own devices.

and updating the corresponding SQTDBL (in case of the accelerator queue pair, this would be SQ1TDBL, whereas for the host-side, the SQ2TDBL). Afterwards, the controller sends a MRd to the Submission queue to fetch the command. The requester (either the DMA Iuventus module or the host software) responds with one or more Cpl TLPs. The controller then proceeds to execute the command. The read command targets either the accelerator's or the host's *Write buffer*, which is specified in the command's PRP1 and PRP2 fields. The NVMe controller can send the data in one or multiple MWrs. Finally, to signal the end of command execution, a CQE is written to the Completion queue. As seen in the same figure, the requester can submit multiple commands which are then processed on the controller. To inform the controller that all completions have been read from the Completion queue, the requester updates the value of the corresponding CQHDBL in controller's registers (similar to the SQTDBL, the accelerator updates the CQ1HDBL value whereas the host the CQ2HDBL value).

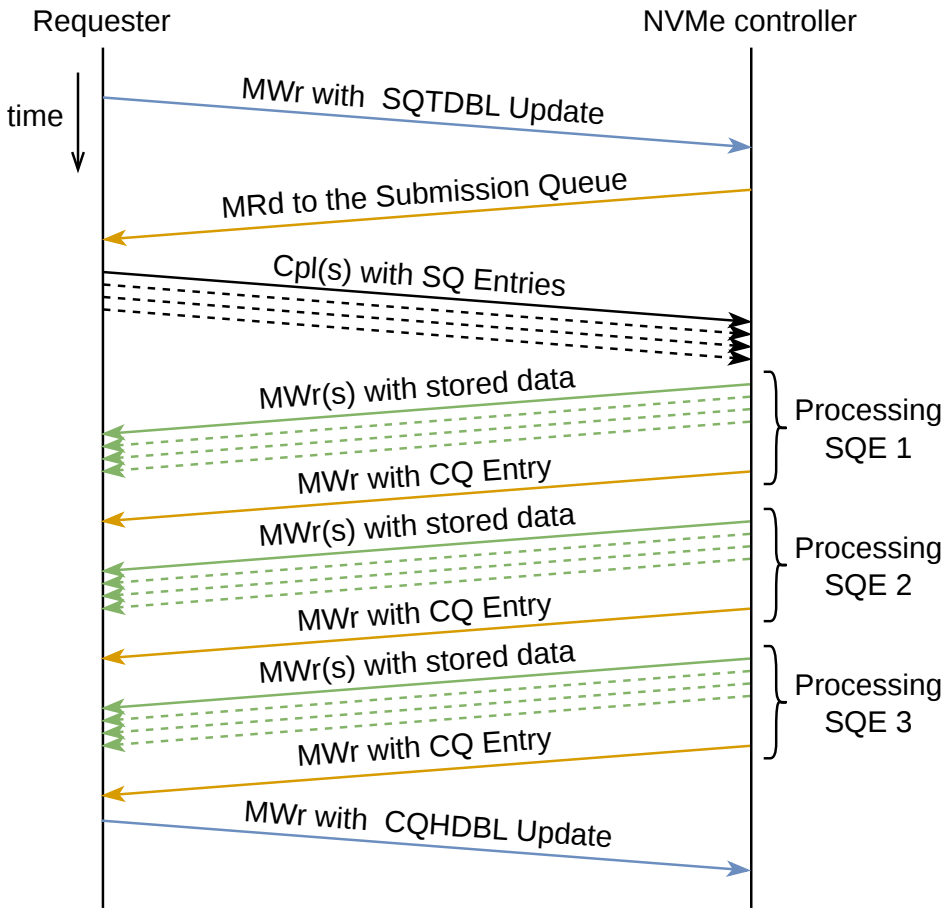

Fig. 4.5: Read command submission. The indexing of SQEs is completely arbitrary since there is no ordering of commands on the NVMe controller's side if multiple commands are fetched.

4.3.2 Write command

The write command contains additional steps compared to the read command as seen in fig. 4.6. The NVMe controller fetches the command(s) in a similar manner, by receiving the SQTDBL update, dispatching the MRd to the requester and receiving completions with SQE data. Afterwards, as a part of command execution, the controller dispatches one or multiple MRds to the requester's *Read buffer* to fetch the data that are received consequently in one or multiple Cpls. The end of command execution is marked by the write of a CQE with the status of the execution. Similarly, the controller can fetch multiple commands and process them as a batch, dispatching the completion every time. The requester then, after processing the CQE, updates the corresponding value of CQHDBL. Optionally, the requester can issue a *Flush* command to permanently store the data on the NVM which can be seen in the same figure as well. This command can be issued on devices that use a separate write cache. It is usually not necessary to explicitly use this command unless synchronization between two requesters is required. The flushing can be left on the automatic mechanism of the controller which also flushes the whole content of a cache when it shuts down.

4.4 Hardware implementation

The hardware architecture enables full P2P communication between an FPGA accelerator and an NVMe device. This architecture is depicted in fig. 4.7. The architecture transports data on streaming buses rather than memory-mapped ones. The only exception is the *Memory Interface* (MI) interface[23] connected to the `Software manager` that contains the C/S registers to configure the DMA module. This operates on memory-mapped chunks of 32 bits. The streaming interfaces, namely `CQ`, `CC`, `RQ`, `Read Data` and `Write Request + Data` follow the *Multi-frame Bus* (MFB) specification[24].

Because of the *Straddling* feature on the PCIe IP's interfaces conforming to the AXI-Stream standard, this protocol had to be arbitrarily extended. This mode splits a transfer[6] into two logical regions where each region can contain a start of a packet. If the first packet ends in the first region, the second packet can begin in the second region, thus effectively increasing throughput for packets that align in such a way and, especially, for small packets that fit in one region. This extension includes adding more subsignals to the `TUSER` while ignoring the `TKEEP` signal since this only works when only one region is utilized.

[6]One transfer corresponds to one bus beat, meaning a clock period when both TVALID and TREADY are high, i.e. both the Transmitter and the Receiver are ready to transfer data[10].

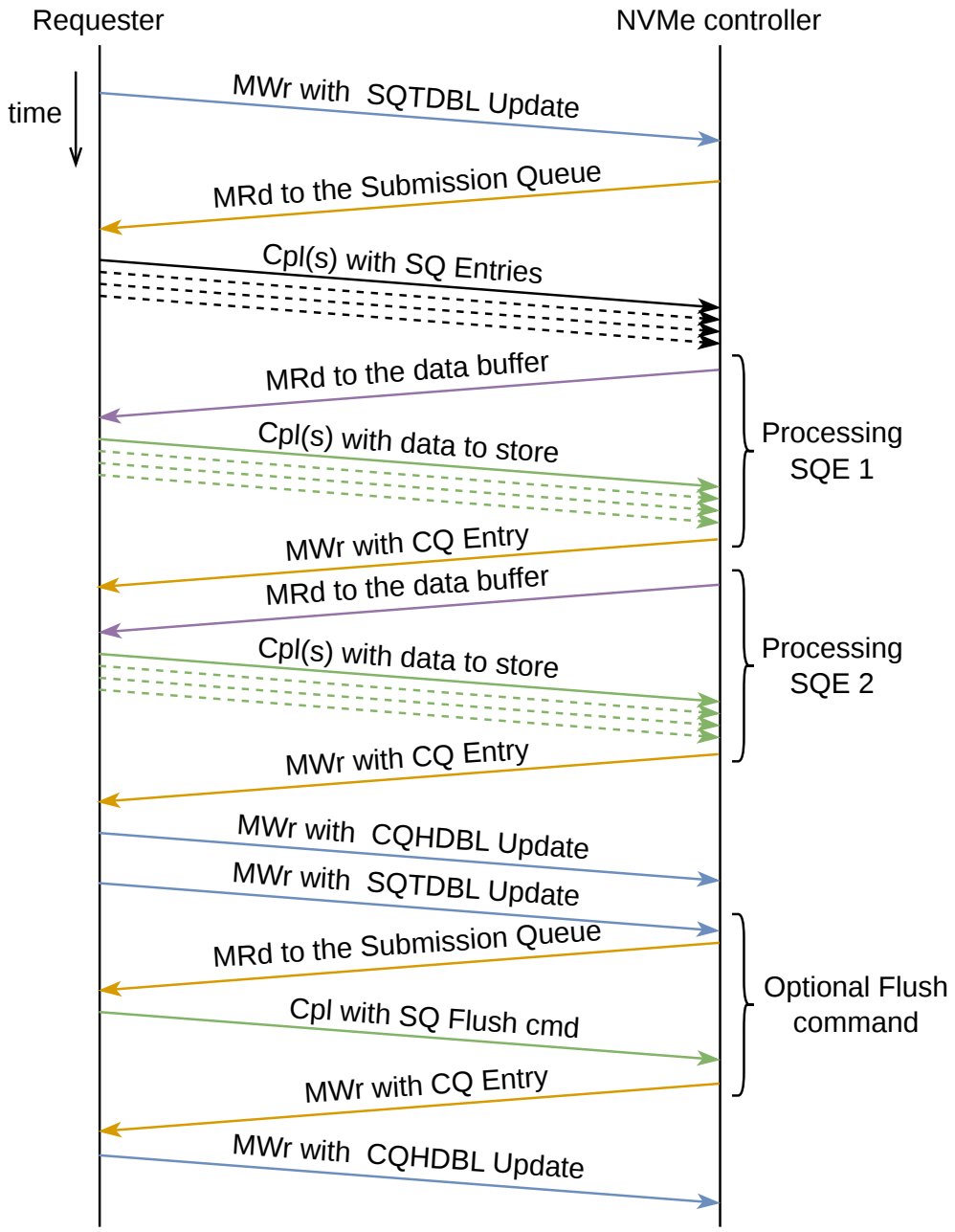

Fig. 4.6: Write command submission. The indexing of SQEs is completely arbitrary since there is no ordering of commands on the NVMe controller's side if multiple commands are fetched.

MFB has been created to exactly address this issue of dispatching multiple frames (or packets) in one clock cycle and does not need additional signals to facilitate that. A transfer consists of multiple *regions*, where each is composed of 1 or more *blocks*. Each block contains then 1 or more *items* and each of the items is n bits long. All of the used interfaces of the PCIe IP have the same configuration, that is 2 regions, 1 block, 8 items and 32 bits[7] per item (referred to in a short form as the *2,1,8,32*

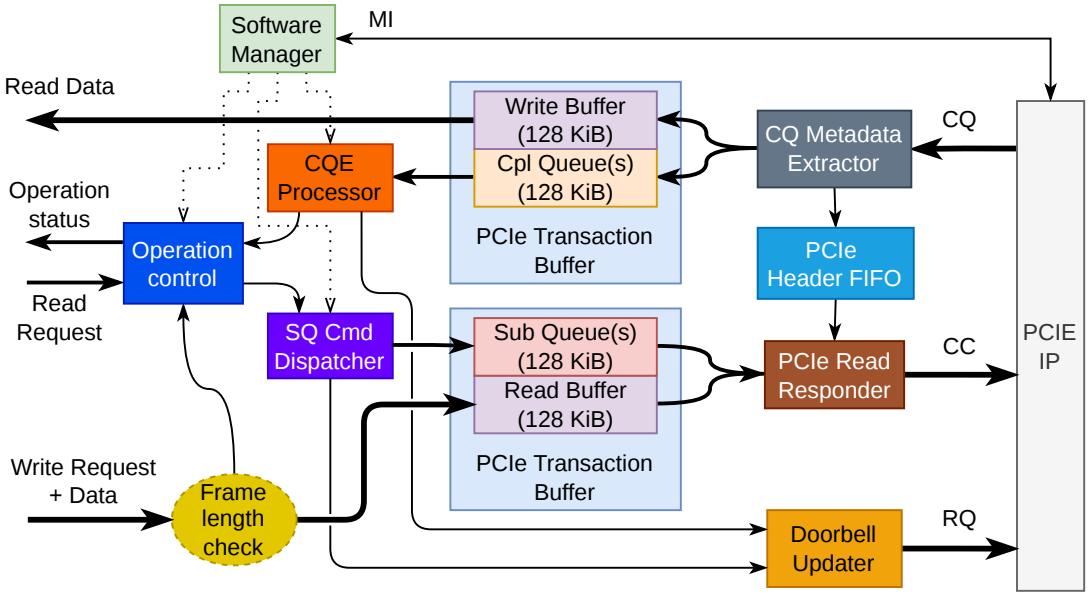

Fig. 4.7: DMA Iuventus block scheme

configuration) resulting in 512 bits. MFB consists of following signals:

DATA containing frame payload.

META containing optional metadata to the frame. The metadata can be aligned either with a SOF of a frame (default) or with an EOF.

SOF indicating which regions of a transfer contain a start of a frame.

EOF indicating which regions contain an end of a frame.

SOF_POS Pointer for every region where the start of a frame is located. A frame can be started with resolution to *blocks*.

EOF_POS Pointer for every region where the end of a frame is located. A frame can be ended with resolution to *items*.

SRC_RDY When high, indicates that a transmitter wants to send data.

DST_RDY When high, indicates the readiness of a receiver to accept data.

The interfaces to the user logic, namely `Read Data` and `Write Request + Data`, follow the MFB specification as well but with a different configuration. Since the highest throughput is no longer required and the smallest size of a memory block that can be requested is 512 B (i.e. one LBA), the configuration has been set to *1,1,64,8* which can be rewired to the standard AXI-Stream bus. All of the streaming buses run on 250 MHz clock provided by the PCIe IP and the same clock is used to drive all other systems of the DMA Iuventus module. The MI interface runs on a slower, 100 MHz clock to relax timing constraints.

[7]Setting the item size to 32 bits is in accord with the PCIe interface operating with DWords as the smallest atomic unit.

4.4.1 Software initialization

The register map for the DMA Iuventus module is listed in chapter B. The component gets configured by the host software by setting the following registers in the `Software manager`:

- `DBL Mask` that serves as a mask of all Doorbell pointers. This mask is determined by the size of the queues.
- `SQTDBL Base Addr H` and `SQTDBL Base Addr L` that contain the address of `SQ1TDBL` in the NVMe BAR.
- `CQHDBL Base Addr H` and `CQHDBL Base Addr L` that contain the address of `CQ1HDBL` in the NVMe BAR.
- `RD Buff Base Addr H` and `RD Buff Base Addr L` that contain the address of the read buffer allocated in the accelerator's BAR 3. This address needs to be supplied by the configuration software since the PCIe IP does not provide an interface to read values in its own BARs of the PCIe Configuration Header.
- `RD Buff PRP List Base H` and `RD Buff PRP List Base L` that contain the base address of the read buffer's PRP List (this list resides in the host memory).
- `WR Buff Base Addr H` and `WR Buff Base Addr L` that contain the address of the write buffer allocated in the accelerator's BAR 2. This address needs to be supplied by the configuration software for the same reason as the address of the read buffer.
- `WR Buff PRP List Base H` and `WR Buff PRP List Base L` that contain the base address of the write buffer's PRP List (this list resides in the host memory).
- `LBA Num Mask` that serves as a limit of the amount of LBAs that can be transferred within a single NVMe command (i.e. the maximum amount that can be assigned to the NLB field in a SQE as specified in the MDTS field in the *Identify Controller* data structure).
- `LBA Space Size H` and `LBA Space Size L` that contain the total amount of usable LBAs on the provided Namespace regardless of if some LBAs have already been written in the past.

Finally, the Iuventus module is activated by assigning bit 0 in the `Control` register to 1 which is acknowledged by the module setting bit 0 in the `Status` register to 1. The acknowledgement happens after all internal components are initialized.

4.4.2 Read command submission

The user logic requests a read from the NVMe device on the `Read Request` interface (Figure 4.7). This is achieved by setting the amount of LBAs to read on the `NVME_RD_REQ_LBA_NUM` and the starting address of the first LBA on `NVME_RD_REQ_LBA_PTR`

inputs. The valid request is confirmed by setting `NVME_RD_REQ_VLD` to high which is acknowledged by the `Operation control` component by asserting `NVME_RD_REQ_RDY`. Because of the hardware constraints explained further, the amount of LBAs that can be requested by the user logic on the `NVME_RD_REQ_LBA_NUM` is capped to 256 (i.e. this signal has a width of 8 bits).

After `Operation control` acknowledges the request, it checks that the request size with its starting pointer does not exceed the Namespace size (thus writing beyond the namespace boundary). The component decides the values of PRP1 and PRP2 fields for the command based on the amount of memory pages that will be transferred. After that, the NVMe Read command is created in the `SQ Cmd Dispatcher` from the provided parameters that were previously assigned to the C/S registers. SQ Cmd Dispatcher maintains an internal FIFO of available CIDs that are assigned to every dispatched command. Reading the status of this FIFO helps to assert the amount of flying commands that are currently executed/waiting on the NVMe controller. This status is available in the C/S registers as the `TAG FIFO Status` field.

The command is then stored in the `Sub Queue` located in the `PCIe Transaction Buffer`. Originally, this buffer has been developed as a part of the DMA Calypte project[25] to store incoming TLPs on the CQ interface. The advantage of this buffer is the ability to store on a link data rate without blocking with address resolution to bytes while also able to store two individual packets (this stems from the two regions the CQ bus is divided into where each packet can address a different location). Because the reading side of the `PCIe Transaction Buffer` has been designed to put data on a streaming bus, this component is used twice in the Iuventus module even when it is underutilized for the Submission queue/Read buffer side. The constraint of capping the amount of LBAs being requested is caused by the maximum size of the memory array in this component which consists of 64 BRAM blocks (since the bus is 64 bytes wide and byte-level addressability is needed). This array has an overall capacity of 256 KiB and it is split into two sectors where one is occupied by the Submission/Completion queue, while the other with the Read/Write buffer. The size of each sector corresponds to exactly 256 LBAs.

As soon as the `SQ Cmd Dispatcher` creates the command and stores it in the Submission queue it sends an update of the SQTDBL to the `Doorbell updater`. This queues this update for a dispatch which is then dispatched whenever the RQ interface is ready. After restart or a long period where the pointer has not been updated, the update is dispatched immediately. After dispatching one update, the delay counter is activated. The pointer can be further updated during this delay but the update is dispatched first when the delay expires. This mechanism has been added to enable submitting multiple commands and then fetching them in batch.

This reduces the pressure on the PCIe interface by lowering the amount of TLPs containing only synchronization data.

The Memory Read Request from the NVMe controller arrives on the CQ interface and is routed from the `CQ Metadata Extractor` to the `PCIe Header FIFO`. The Metadata extractor calculates the exact byte address and splits TLPs based on the BAR and the request type. The header FIFO receives all of the read requests that can target either the Read Buffer or the Submission Queue. The read requests are processed by the `PCIe Read Responder`, which is able to read data of arbitrary length from the transaction buffer and put them on the CC interface. If the size of data exceeds 128 B, the payload gets segmented into 128-byte chunks[8] which are dispatched as separate Cpl TLPs, effectively utilizing the *Dword count* and *Byte count* fields in the CC TLP header. A single command in a queue is usually read by just one MRd from the NVMe controller.

After some delay, the controller sends data from the NVM that had been requested in the processed command. These data are routed towards and written to the `Write Buffer`. Finally, the controller sends a CQE with the status of the operation. The `CQE Processor` component detects the received CQE based on the current value of the CQHDBL and the *Phase tag*. The component then returns the status of the command to the `Operation control` and updates the value of CQHDBL[9]in the `Doorbell updater`. When the returned status is successful, the `Operation control` then reads the data from the `Write buffer`, thus returning them to the user logic.

For software-facing reporting, the command status is returned on the `Operation status` interface. When the `OP_STAT_TYPE` signal on this interface is set high, the status of the last read request is returned, otherwise, when set to low, the status of the last write request is returned. For the accurate distinction of status, the `OP_STAT_CODE` signal outputs the status of the operation, which is listed in table 4.1. A valid status is signalized by setting the `OP_STAT_VLD` output high for a single clock cycle.

4.4.3 Write command submission

A write request is initiated by directly writing a packet on the `Write Request+Data` interface which is a MFB bus with the `META` signal containing a pointer of the LBA to which the packet data should be written. This pointer travels with the start of

[8]Segmenting data into chunks of this size simplifies the design. Otherwise, the MPS value would have to be consulted with the PCIe IP and the size of the segmented blocks would have to be dynamically configurable since the value of MPS can range from 128 B to 4096 B. Setting the size to the smallest possible value enables the use of the static setting for segmentation.

[9]The values of Doorbell pointers can be found in the registers listed in chapter B. The value of CQTDBL is not provided since it is internal to the NVMe controller and it is not necessary for the correct processing of CQEs.

Table 4.1: Operation status codes as output by DMA Iuventus

Code	Description
0b00	Success
0b01	General fault (more information available in status registers)
0b10	LBA Out of Range
0b11	Reserved for future use

every packet. The Iuventus module checks the length of an incoming packet which is provided for the `Operation control` after the packet ends while the packet data have been written to the *Read buffer*. The packet should not be longer than 128 KiB in order to fit in the buffer. If a packet is longer than this, it gets truncated to fit in and the rest of its data is dropped.

After receiving a packet's length, the `Operation control` checks if the amount of data that are going to be written to the NVMe on the specified LBA does not cross the namespace boundary. When this check passes, the component submits an NVMe Write command in the same way as the previously described submission of the read command, i.e. by writing the command to the Submission queue and then updating the SQTDBL in the `Doorbell updater`. The NVMe controller reads that command from the Submission queue whose data are then returned on the CC interface. During execution of this command, the controller sends multiple read requests to the read buffer that are routed from the CQ interface through the `CQ Metadata Extractor` to the header FIFO. These requests can request data chunks as large as the current MRRS setting (up to 4 KiB) but the `PCIe Read Responder` slices each of these requests into 128 B chunks as previously described.

The completion of a command happens in the same way as in case of the read command where the NVMe controller writes a CQE into the Completion queue which is then detected by the `CQE Processor` and its status returned to the `Operation control`[10]. The status is also signalized on the `Operation status` interface with the same status codes as described in table 4.1.

[10]The flushing operation after a write has not been implemented because this thesis only demonstrates the concept of the shared access between a CPU and an FPGA accelerator without focusing on data race. However, the operation can be implemented by extending SQE format generation in SQ Cmd Dispatcher component and by extending the state machine in the Operation control component.

5 Filesystem integration

The NVMe userspace driver provided within the SPDK framework[20] with its API provides all necessary functions for NVMe device configuration and status management[1]. The API also provides functions, like `spdk_nvme_ns_cmd_read` and `spdk_nvme_ns_cmd_write` that can submit read/write NVMe commands and operate on the raw data stored on the NVM. However, operation on the raw sectors of LBAs cannot be easily used in any OS since it introduces a large amount of device-specific code. This thesis aims to share a running (but also standardized) filesystem on the NVMe drive between a host and an FPGA accelerator.

Linux-based operating systems distinguish three types of devices: *character (char)* devices that operate with their data in streams (like a sound card or serial console), *network* devices that serve only as an interface with callbacks returning/putting packet data; and *block (bdev)* devices that operate on large blocks of data (usually 512 B or more). Both the char and block devices are available as file entries in the operating system (i.e. device files in the `/dev` directory), although the block devices are the only ones available to be read back and forth as a normal file (meaning the users are able to perform seek on them). There can be exceptions to some char devices but they generally prohibit to perform a seek to their device files. The block devices are the only ones that are available to run a file system[18] because they provide an abstraction over multiple different storage hardware by disposing of a unified set of operations[16]. The Linux filesystem drivers then attach to specific block devices.

An interesting case occurs when a userspace framework is used to operate on a block device such as the SPDK application operating on an NVMe drive in case of this thesis. A filesystem can be mounted only if it is visible by the kernel, which is where *Userspace block device* (ublk) comes into scene. This is a high-performance driver that allows attaching a userspace application (called *ublk server*) as a kernel-based bdev. The ublk framework profits from `io_uring` as a recent addition to the Linux kernel that introduces a fast way of userspace-kernel communication which is fully asynchronous and utilizes a pair of queues similar to the ones specified in NVMe (that is *Submission queue* and *Completion queue*). This way allows to submit multiple requests to the kernel or vice versa and then just wait for the completion of such batch (works as a part of the asynchronous I/O contrary to the traditional way

[1]The driver does not use interrupts since these cannot be routed to the userspace and they introduce a significant overhead due to forced context switches. Therefore, they have been disabled for the NVMe drive and for the FPGA accelerator as well. The NVMe standard supports wide range of interrupt types used in the PCIe. These interrupts can be used to notify the requester that new entries arrived to the completion queue. The SPDK framework incorporates polling to wait for devices which helps to reduce latency for the cost of putting some load to the CPU running polling loops[20].

which uses system calls that are blocking and only one can be active at a time)[26]. The scheme of how this is configured is depicted in fig. 5.1. The ublk server asks the ublk driver to create one or more ublk devices (located in `/dev/ublkb*`). One device of that sort is then formatted as a filesystem and mounted. Every operation on this filesystem (like read, write or flush) is forwarded through the ublk driver back to the server where it is processed and completed[27, 28].

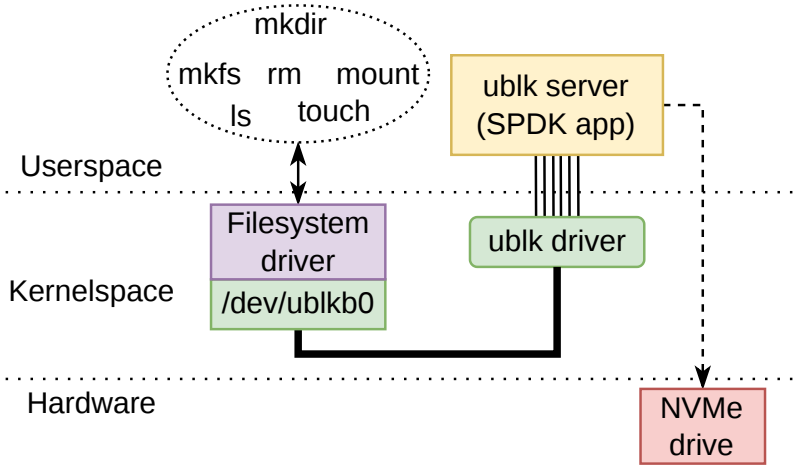

Fig. 5.1: The block scheme of the initialized application proposed in this thesis running one ublk device.

The *extensible File Allocation Table* (exFAT) filesystem has been chosen for the demonstration because of its ease of implementation, ability to store very large files, and support for very large storages. This filesystem is also supported by the Linux kernel, thus by available tools, like `mkfs`, `dumpfs` or `lsblk`. The portion of a filesystem that can store user data (the s.c. *Cluster Heap*), is organized as an array of *clusters* where each consists of multiple *sectors*, each, in case of the NVMe drive, of a size of one LBA[29]. The configuration used in this thesis uses the cluster size of 128 KiB which corresponds to 256 LBAs (this cluster size is automatically set by the `mkfs.exfat` utility[30]). The important features of the standard are the following :

- The allocation of clusters is not kept inside the nodes of the *File Allocation Table* (FAT) (as it is in the case of the FAT32 filesystem) but in the *Allocation Bitmap* located in the *root cluster* of Cluster Heap.
- The nodes of the FAT are still 32 bit pointers. This explains the increasing size of a cluster in order to encompass large storage media.
- A file does not have to use the FAT if such file can be stored in a contiguous string of clusters. This is marked by an attribute of the file and the amount of utilized clusters is still determined by a size of the file.

- The specification allows to encompass storage volumes as large as 64 ZB. However, it is recommended to not use more than $2^{24} - 2$ clusters in the cluster heap which results in a maximum capacity of 512 TB given the maximum cluster size of 32 MB[29].

6 Evaluation

The evaluation was carried out on the previously mentioned *AMD Alveo U55C* acceleration card[9] using Vivado 2025.1 for synthesis. For software demonstration, a custom server has been used running AMD Threadripper PRO 5955WX with 16 cores at 4.579 GHz as the CPU. The system disposes of 128 GB of RAM and it allows up to 7 Gen4x16 PCIe cards to be simultaneously connected. Each of the PCIe connectors also supports bifurcation. The acceleration card has been configured as *Gen4x8 Endpoint* with 2 Functions. The NVMe device tested in this work has been the *SK Hynx PC611* with 512 GB stated capacity, running as Gen3x4 Endpoint and conforming to the NVMe v1.3 specification.

6.1 Resource consumption

The consumption of FPGA resources has been summarized in table 6.1. All of these results are taken *post-implementation* after the design has been fully placed and routed. The resource utilization for the whole design is shown in the *Test design total* column. Together with the DMA module and the PCIe IP, the test design contains several additional components:

- The logic connecting all configurable components with the MI bus that sits next to the PCIe IP. The configuration interconnect is organized as a tree and allows the access to the C/S registers of required components.
- Memory for the PCIe with *Extended configuration space* that contains the *DeviceTree*[31] structure with nodes that can be accessed by the `nfb` driver for register access.
- Access to the temperature sensor through the *SYSMON* primitive[32].
- The boot logic allowing to load an FPGA bitstream through PCIe into the internal flash and trigger design reload through *Internal Configuration Access Point* (ICAP)[33].
- Testing logic to dispatch requests to the DMA Iuventus.
- Measurement component for NVMe request latency with its storage for a histogram.
- One *Integrated Logic Analyzer* (ILA) core[34] to show transactions on the `RD_MFB_*` interface of the DMA module.

The design meets timing requirements with 0.079 ns WNS and 0.010 ns WHS.

Table 6.1: FPGA resource consumption for various components of design used in this thesis

Resource type	DMA Iuventus		PCIe IP		Test design total	
	abs	%	abs	%	abs	%
LUT	14625	1.12	7661	0.59	30502	2.34
LUTRAM	268	0.04	745	0.12	3861	0.64
FF	23645	0.91	18429	0.71	67106	2.57
BRAM Tile	134.5	6.67	22	1.09	253	12.55
URAM	0	0	0	0	11	1.15

6.2 Latency measurement

The latency has been measured for the requests originating in the user logic on the DMA Iuventus interface by a custom component created in the `USER_CORE` entity (files `user_core_ent.vhd` and `user_core_test_arch.vhd`). The measurement circuit consists mainly of a data logger and a latency meter. The latency meter starts measuring from the dispatch of either a read request or a write request (remember that, at every point in time, there can be only one flying request) and ends the measurement after the operation is completed, i.e. when `OP_STAT_VLD` pulses high. The width of a timestamp has been set to 2^{28} which allows to capture delays over 1 s. The valid latency value is stored by the `DATA_LOGGER` component that stores maximum, minimum and a histogram of captured values. The data logger stores only 22 bits of a timestamp value from the latency meter since this should be sufficient according to [16, 35]. There is a detection of the timestamp overflow that would mess with the values already stored in the data logger in which case the whole measurement has to be repeated. The histogram consists of 2^{15} bins that give it a resolution of 512 ns. The measurement parameters were varied based on multiple variables:

- Operation type: read or write
- Access pattern: sequential or random addressing
- Selection of request sizes in LBAs: 1, 8, 64 or 256 being copied

Each measurement has been repeated 50000 times. The sequential addressing starts at address number 0 and advances in increments equal to the size of the request. The random addressing is actually a pseudo-random sequence created by a LFSR. The results are listed in table 6.2. The maximal and minimal values are the only ones taken exactly from the data logger. The quantiles are taken from linearly interpolated cumulative distribution function derived from the histogram. The read operation is a more expensive one since the NVMe drive has to fetch the data from the NVM. An obvious fact occurs that the bigger the request's size, the bigger the

latency. Surprisingly however, the latency increases only sublinearly indicating that there is some optimization for fetching of large data on the NVMe's side. With regard to the read operation, there has not been observed any substantial difference between access patterns.

Table 6.2: Results of latency measurement for different operations, request sizes and access pattern (µs)

Op.	**Access**	**Size** [LBAs]	**Min**	P_1	P_{50}	P_{80}	P_{99}	**Max**
RD	Random	1	46.64	50.32	66.71	78.65	81.75	1952.72
		8	47.81	65.54	70.30	80.40	82.88	1840.14
		64	86.90	111.77	130.60	143.96	149.33	1665.92
		256	143.27	180.77	203.46	220.98	238.26	357.50
	Sequential	1	62.12	62.55	63.15	64.83	65.97	173.30
		8	47.80	48.73	67.89	80.66	82.97	1951.48
		64	85.98	111.33	130.51	143.87	148.51	2029.71
		256	144.12	178.78	201.50	218.77	237.61	1904.40
WR	Random	1	8.34	8.28	8.61	9.07	107.28	5379.27
		8	9.50	9.77	10.26	26.65	88.71	5385.86
		64	19.70	19.69	20.41	23.91	139.18	5633.68
		256	54.05	54.51	62.65	69.77	454.14	6692.82
	Sequential	1	8.34	8.24	8.45	8.60	8.70	10.19
		8	9.56	9.77	10.18	64.46	301.59	5949.84
		64	19.71	19.65	20.44	78.26	405.75	6499.20
		256	54.10	58.40	71.57	90.12	251.26	5860.76

The write operation has been observed to be visibly faster than the read operation. The cause of this may be twofold: Firstly, the measurement does not perform `FLUSH` operation after a write is issued and, secondly, the tested NVMe drive contains an on board cache for data that are about to be stored to the NVM. Storing all of the copied data on device's cache is already considered as a completed operation which results in a faster completion receive and lower latency.

6.3 Application usage

This section describes how the application `fpga_zero_copy` that provides shared CPU-FPGA filesystem access should be used. The source files for this are located in the SPDK repository under `examples/nvme/fpga_zero_copy`. The guide presumes that the user finished the installation guide in chapter A and the executables are built in the `build/` directory. The installation guide describes the unbinding of

the NVMe drive on the BDF `0000:61:00.0` and the FPGA acceleration cards's function 1 on `0000:41:00.1`. These devices have the following BAR configuration:

Listing 6.1: Looking for BAR configuration of unbound devices

```
$ sudo lspci -v -s 0000:61:00.0 ; sudo lspci -v -s 0000:41:00.1
61:00.0 Non-Volatile memory controller: SK hynix PC611 NVMe Solid
   State Drive (prog-if 02 [NVM Express])
        Subsystem: SK hynix PC611 NVMe Solid State Drive
        Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV-
           VGASnoop- ParErr- Stepping- SERR- FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast
           >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
        Latency: 0, Cache Line Size: 64 bytes
        Interrupt: pin A routed to IRQ 97
        Region 0: Memory at b7200000 (64-bit, non-prefetchable)
           [size=16K]
        Region 2: Memory at b7205000 (32-bit, non-prefetchable)
           [size=4K]
        Region 3: Memory at b7204000 (32-bit, non-prefetchable)
           [size=4K]

    ... some output omitted ...

        Kernel driver in use: uio_pci_generic
        Kernel modules: nvme

41:00.1 Memory controller: Cesnet, z.s.p.o. Device c020
        Subsystem: Cesnet, z.s.p.o. Device c020
        Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV-
           VGASnoop- ParErr- Stepping- SERR- FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast
           >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
        Latency: 0, Cache Line Size: 64 bytes
        Region 0: Memory at b5060000 (32-bit, non-prefetchable)
           [size=128K]
        Region 1: Memory at b5040000 (32-bit, non-prefetchable)
           [size=128K]
        Region 2: Memory at b5020000 (32-bit, non-prefetchable)
           [size=128K]
        Region 3: Memory at b5000000 (32-bit, non-prefetchable)
           [size=128K]

    ... some output omitted ...

        Kernel driver in use: uio_pci_generic
```

The devices listed are correctly configured which can be seen by enabled memory

transactions (attribute `Mem+`), permitted bus-mastering (attribute `BusMaster+`) and disabled pin-based interrupts (attribute `DisINTx+`). The BAR configurations are marked as `Region N` where the FPGA accelerator contains the previously initialized 4 BARs and the NVMe drive the needed BAR 0 (other BARs contain vendor-specific registers). The devices are also correctly bound to the `uio_pci_generic` driver as present in the `Kernel driver in use:` field.

The `fpga_zero_copy` application is built in SPDK under the `build/examples` directory and executed in the following way (the size of a queue is just a placeholder since the DMA Iuventus core allows only one flying request at a time):

Listing 6.2: Executing the SPDK application

```
# Running the application on the specified SSD and on the implicit
    NFB device (index 0)
$ sudo ./fpga_zero_copy -t 'traddr:0000:61:00.0' -q8
Initializing NVMe Controller for device 0000:61:00.0
Probing 0000:23:00.0 ...
Probing 0000:2d:00.0 ...
Probing 0000:61:00.0 ...
Probing 0000:62:00.0 ...
Attaching to 0000:61:00.0 ...
Controller options:
        Number of IO queues:   16
        Size of IO queues:     8
        IO queue requests:     8
        NS 1 size:             512GB
        NS number of sectors:  1000215216
        NS sector size:        512B
        LBA Mask:              x1ff (511)
Controller CSTS register:
        CSTS.RDY: 1
        CSTS.CFS: 0
        CSTS.SHST: 0
        CSTS.NSSRO: 0
        CSTS.PS: 0
Physical address of NVME BAR0: 0xb7200000
Doorbell stride: 0
PCIE domain initialization complete
FPGA PCIe device context:
        SQ VADDR: 0x201080088000
        SQ PADDR: b5060000
        SQ size:  131072 bytes
        CQ VADDR: 0x2010800a8000
        CQ PADDR: b5040000
        CQ size:  131072 bytes
        WRBUFF VADDR: 0x2010800c8000
```

```
        WRBUFF PADDR: b5020000
        WRBUFF size:  131072 bytes
        RDBUFF VADDR: 0x2010800e8000
        RDBUFF PADDR: b5000000
        RDBUFF size:  131072 bytes
Default queue pair options:
IO queue size: 8
IO queue requests: 8
Allocate qid 1
HW CQ allocated!
HW SQ allocated!
        SQTDBL physical address: 0xb7201008
        CQHDBL physical address: 0xb720100c
Queues allocated.
Write PRP List (vaddr: 0x20007feeb000, paddr: 0x1dffeeb000):
    ... list entries omitted ...
Read PRP List (vaddr: 0x20007fee7000, paddr: 0x1dffee7000):
    ... list entries omitted ...
Host filesystem bdev registered as: /dev/ublkb0
  To format: mkfs.ext4 /dev/ublkb0
  To mount:  mount /dev/ublkb0 /mnt/nvme
Initialization complete. Starting main loop.

# The app then blocks the terminal when the main loop starts and
   only waits for SIGINT signal (normally bound to Ctrl-C)
```

The `fpga_zero_copy` puts out valuable information about the configured parameters. It probes all unbound NVMe drives in the system but attaches only our chosen one. The BAR addresses of the NVMe as well as of the FPGA accelerator. The values of BARs are assigned to the FPGA context as mentioned previously in the implementation chapter, i.e. BAR 0 address to the Submission Queue, BAR 1 address to the Completion Queue, BAR 2 address for the write buffer and BAR 3 address for the read buffer. The assignment is followed by allocating *Queue Identifier* (QID) of 1 for the queues in the hardware and attaching these queues to the NVMe controller. The doorbell register offsets needed to be calculated from the offset in BAR 0 of the NVMe combined with *Doorbell stride*. These offsets are also written to the FPGA context, meaning into the configuration registers of the DMA Iuventus module. The PRP Lists are initialized afterwards and their physical addresses are published to the FPGA context as well. Lastly, the ublk device is created as mentioned in the output and published as `/dev/ublkb0` with examples of how to format and mount it.

The device context can be verified by separate tools available in the downloaded NDK-FPGA repository. These are located in the `apps/iuventus/sw` directory.

Listing 6.3: Verifying NFB device context

```
(my-python-env) $ ./iuventus_reg_status.py
ctrl_reg               : ENABLE: 1
                         |  RPT_UPD_EN: 1

status_reg             : RDY: 1
                         | RST_DONE: 1
                         | TAG_INIT_DONE: 1

sqtdbl                 : 0               (0x0)
sqhdbl                 : 0               (0x0)
cqhdbl                 : 0               (0x0)
dbl_mask               : 0x7     (7)
sqtdbl_baddr           : 0xb7201008
cqhdbl_baddr           : 0xb720100c
rdbuff_baddr           : 0xb5000000
rdbuff_prp_list_ptr    : 0x1dffee7000
wrbuff_baddr           : 0xb5020000
wrbuff_prp_list_ptr    : 0x1dffeeb000
last_cq_entry          : ('cmd_specific: 0 (0x0), rsv1: 0 (0x0),
   sqhdbl: 0 (0x0), sq_id: 0 (0x0), '
                           'cmd_id: 0 (0x0), phase_tag: 0 (0x0),
                              stat_code: 0 (0x0), stat_code_type:
                              0 '
                           '(0x0), rsv2: 0 (0x0), more: 0 (0x0),
                              do_not_retry: 0 (0x0)')
sqes_dispatched        : 0
cqes_processed         : 0
pcie_rds               : 0
pcie_rds_bytes         : 0
pcie_wrs               : 8
pcie_wrs_bytes         : 8
sq_pcie_rds            : 0
sq_pcie_rds_bytes      : 0
lba_mask               : 511             (0x1ff)
succ_cpls              : 0
unsucc_cpls            : 0
err_mask               : 0
rdbuff_pcie_rds        : 0
rdbuff_pcie_rds_bytes  : 0
wrbuff_pcie_wrs        : 0
wrbuff_pcie_wrs_bytes  : 0
lba_space_size         : 0x3b9e12b0       (1000215216 blocks =
   476.94 GB)
cq_pcie_wrs            : 8
cq_pcie_wrs_bytes      : 8
cqhdbl_reg_upd         : 0
```

```
cqhdbl_rpt_upd         : 15
sqtdbl_reg_upd         : 0
sqtdbl_rpt_upd         : 15
meta_ptr               : 0x0
nvme_rd_cmd_bytes      : 0
nvme_wr_cmd_bytes      : 0
wrbuff_usr_rds         : 0
wrbuff_usr_rds_bytes   : 0
rdbuff_disp_rds        : 0
rdbuff_disp_rds_bytes  : 0
sq_disp_rds            : 0
sq_disp_rds_bytes      : 0
tag_fifo_status        : 2048/2048
```

The `iuventus_reg_status` tool actually prints all currently stored values in the configuration registers of the DMA Iuventus module as defined in chapter B. We see that FPGA context has been written correctly into all `*_baddr` as well as `*_prp_list_ptr` fields. The component is now able to process requests to the NVMe drive which is signalized by `ENABLE` and `RDY` fields both set to 1. The capacity of the namespace is stored in the `lba_space_size` while the maximum amount of data that can be transfered within one command (i.e. the value of MDTS field of the *Identify controller* data structure) is copied to the `lba_mask` field. The core has not processed any commands yet. First we format the created ublk device.

Listing 6.4: Formatting the created ublk device

```
# The block device can be formatted
$ sudo mkfs.exfat /dev/ublkb0
exfatprogs version : 1.3.2
Creating exFAT filesystem(/dev/ublkb0, cluster size=131072)

Writing volume boot record: done
Writing backup volume boot record: done
Fat table creation: done
Allocation bitmap creation: done
Upcase table creation: done
Writing root directory entry: done
Synchronizing...

exFAT format complete!

# The formatted drive can then by verified (the most important are
    the Root entries)
$ sudo dump.exfat /dev/ublkb0
exfatprogs version : 1.3.2
-------------- Dump Boot sector region --------------
```

```
Volume Length(sectors):                  1000215216
FAT Offset(sector offset):               2048
FAT Length(sectors):                     30524
Cluster Heap Offset (sector offset):     32768
Cluster Count:                           3906962
Root Cluster (cluster offset):           7
Volume Serial:                           0xfdc7fd63
Bytes per Sector:                        512
Sectors per Cluster:                     256

----------------- Dump Root entries -----------------
Volume label entry position:             0x10a0000
Volume label character count:            0
Volume label:
Upcase table entry position:             0x10a0060
Upcase table start cluster:              6
Upcase table size:                       5836
Bitmap entry position:                   0x10a0040
Bitmap start cluster:                    2
Bitmap size:                             488371

---------------- Show the statistics ----------------
Cluster size:                            131072
Total Clusters:                          3906962
Free Clusters:                           3906956
```

The system can now be mounted and operated as a normally attached drive. To make the created files permanently stored is made by running the `sync` command which not only flushes the cache on the NVMe drive but also the catche maintained by the filesystem driver in the kernel.

Listing 6.5: Formatting the created ublk device

```
# Mounting with relaxed privileges for the non-superuser (uid and
    gid numbers can differ)
$ sudo mount -o uid=1000,gid=1000,umask=000 /dev/ublkb0
    /mnt/nvmexp/

# Create some experimentary files
$ cd /mnt/nvmexp
$ echo "NCT␣rules!" > tst1.txt
$ echo "Heidelberg␣is␣a␣nice␣town." > tst2.txt
$ mkdir my_dir
$ echo "Hidden␣file␣in␣one␣directory" > my_dir/tst1.txt

# Creating large contiguous files is generally problematic for
    exFAT but this may work
$ dd if=/dev/zero of=./large_file.txt bs=1M count=1
```

```
# Permanently store the written files on the media
$ sync /dev/ublkb0
```

We can return back to the software tools in the NDK-FPGA repository. Besides the `iuventus_reg_status.py` there is another app as a placeholder for a custom logic in the FPGA that creates requests to the NVMe drive[1]. Be careful with writing operations when you have a mounted filesystem or other useful files on the NVMe drive since you can corrupt them.

Listing 6.6: Creating requests from the FPGA

```
# Reading one sector (0-based value) from the LBA 0
(my-python-env) $ ./iuventus_rw_test.py -r 0 0

# Writing 256 sectors to LBA 475
(my-python-env) $ ./iuventus_rw_test.py -w 475 255

# Single latency measurement of write operation with 10000
   repetitions, random addressing and 46 sectors in size
(my-python-env) $ ./iuventus_rw_test.py -l wr 10000 rand 45

# Measuring latency with parameter sweep as measured in the
   previous section and writing these results into a graph
(my-python-env) $ ./iuventus_rw_test.py -mp
```

The internal status of the DMA Iuventus can be checked every time by the `iuventus_reg_status.py` tool.

Listing 6.7: Example of DMA Iuventus status

```
(my-python-env) $ ./iuventus_reg_status.py
ctrl_reg               : ENABLE: 1
                         |  RPT_UPD_EN: 1

status_reg             : RDY: 1
                         | RST_DONE: 1
                         | TAG_INIT_DONE: 1

sqtdbl                 : 0               (0x0)
sqhdbl                 : 0               (0x0)
cqhdbl                 : 0               (0x0)
dbl_mask               : 0x7     (7)
sqtdbl_baddr           : 0xb7201008
cqhdbl_baddr           : 0xb720100c
rdbuff_baddr           : 0xb5000000
```

[1]For more information, see the help of the application that is printed by the -h switch of the application

```
rdbuff_prp_list_ptr    : 0x1dffee7000
wrbuff_baddr           : 0xb5020000
wrbuff_prp_list_ptr    : 0x1dffeeb000
last_cq_entry          : ('cmd_specific: 0 (0x0), rsv1: 0 (0x0),
   sqhdbl: 0 (0x0), sq_id: 1 (0x1),'
                         'cmd_id: 132 (0x84), phase_tag: 0 (0x0),
                            stat_code: 0 (0x0), stat_code_type: '
                         '0 (0x0), rsv2: 0 (0x0), more: 0 (0x0),
                            do_not_retry: 0 (0x0)')
sqes_dispatched        : 80000
cqes_processed         : 80000
pcie_rds               : 1040000
pcie_rds_bytes         : 250880000
pcie_wrs               : 11280008
pcie_wrs_bytes         : 718080008
sq_pcie_rds            : 80000
sq_pcie_rds_bytes      : 5120000
lba_mask               : 511              (0x1ff)
succ_cpls              : 80000
unsucc_cpls            : 0
err_mask               : 0
rdbuff_pcie_rds        : 960000
rdbuff_pcie_rds_bytes  : 245760000
wrbuff_pcie_wrs        : 11200000
wrbuff_pcie_wrs_bytes  : 716800000
lba_space_size         : 0x3b9e12b0       (1000215216 blocks =
   476.94 GB)
cq_pcie_wrs            : 80008
cq_pcie_wrs_bytes      : 1280008
cqhdbl_reg_upd         : 80000
cqhdbl_rpt_upd         : 1047
sqtdbl_reg_upd         : 80000
sqtdbl_rpt_upd         : 1047
meta_ptr               : 0x0
nvme_rd_cmd_bytes      : 716800000
nvme_wr_cmd_bytes      : 245760000
wrbuff_usr_rds         : 50000
wrbuff_usr_rds_bytes   : 716800000
rdbuff_disp_rds        : 960000
rdbuff_disp_rds_bytes  : 245760000
sq_disp_rds            : 80000
sq_disp_rds_bytes      : 5120000
tag_fifo_status        : 2048/2048
```

The transmission was successful since there are no `unsucc_cpls` and no errors logged in the `err_mask`.

Before killing the running application, the block device needs to be unmounted. The `fpga_zero_copy` application can be killed by sending the `SIGINT` signal to it which is normally bound to Ctrl-C keybinding. This triggers a shutdown process of all devices in the chain with correct disabling of the DMA Iuventus module.

Conclusion

This thesis introduced the solution for efficient, directly addressable access to the NVMe from the FPGA accelerator as an alternative to traditional staging of data in the host memory. The proposed implementation demonstrated shared CPU-FPGA filesystem access to an NVMe device using PCI Express peer-to-peer communication, with the CPU primarily responsible for configuration and standard filesystem operation. The core objective was to reduce avoidable software overhead that the copying of data involves as well as the NVMe command creation and memory-bus pressure while preserving practical integration with a Linux software stack.

To achieve this, the work combined hardware and software co-design. On the hardware side, the DMA Iuventus IP core was developed and integrated into the FPGA design to process NVMe-related PCIe traffic, maintain queue state, and serve requests from accelerator-mapped queue and buffer regions. On the software side, SPDK with Linux ublk was used to create a userspace control path to a mountable block device, enabling validation with a standard filesystem workflow instead of only raw block-level testing. This architecture established an operational path for host-bypassing data exchange between the FPGA and NVMe controller while preserving host access to shared storage.

The implementation was validated on the AMD-based computer architecture equipped with the *AMD Alveo U55C* platform and an NVMe SSD. The reported measurements and functional tests confirmed correct operation of command submission and completion handling, successful read/write behavior for both the CPU and accelerator access paths, and latency characteristics consistent with expected NVMe operation. Resource utilization and timing results showed that the proposed hardware is implementable on a practical data-center accelerator.

At the same time, the solution has limitations emerging from its current scope. The presented setup was evaluated on a specific hardware and firmware environment. The implementation choices (like IOMMU settings or only one flying command from the DMA Iuventus module) prioritize functionality and experimental control over multi-tenant or high throughput use-cases.

Future work can therefore focus on strengthening deployability and scaling. Promising directions include operation with enabled IOMMU where `vfio` driver can be used to access the unbound devices for non-superusers, extension toward more parallel queue configurations including queues with larger size that would allow more flying commands from the hardware, or other models of NVMe drives. The testbed used in this thesis provided 4 different SSDs but only some of them worked at a time in the communication with the hardware (only 1 at the time when this thesis was written). Although all of these NVMe drives were normally operable

from the host system, the FPGA access to all of them was not possible all of the time. The cause of this remains largely undiscovered.

Bibliography

[1] Mike Jackson and Ravi Budruk. *PCI Express Technology*. Mindshare, Inc., 2012. ISBN: 978-0-9836465-2-5.

[2] PCI-SIG. *PCI Express Base Specification*. Revision 4.0, Version 1.0. PCI-SIG, 2017.

[3] Wikipedia contributors. *PCI Express — Wikipedia, The Free Encyclopedia*. [Online; accessed 13-March-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=PCI_Express&oldid=1337274114.

[4] Al Yanes. *An Introduction to Form Factors for PCI Express*. URL: https://pcisig.com/introduction-form-factors-pci-express%C2%AE. [Online; accessed: 23-October-2025].

[5] Intel Corporation. *ATX Version 3.0 Multi Rail Desktop Platform Power Supply*. Version 2.0. 2022. URL: https://www.cybenetics.com/attachs/52.pdf. [Online; accessed: 4-March-2026].

[6] Wikimedia Commons. *Example of PCI-E x4, x16, x1 and 2nd x16*. URL: https://commons.wikimedia.org/wiki/File:PCI-E_%26_PCI_slots_on_DFI_LanParty_nF4_SLI-DR_20050531.jpg. [Online; accessed: 23-October-2025].

[7] Mike Anderson. *The IOMMU Impact: I/O Memory Management Units*. URL: https://medium.com/@mike.anderson007/the-iommu-impact-i-o-memory-management-units-feb7ea95b819. [Online; accessed: 23-October-2025].

[8] Red Hat Virtualization Documentation Team. *Hardware Considerations for Implementing SR-IOV*. 2018. URL: https://docs.redhat.com/de/documentation/red_hat_virtualization/4.2/html/hardware_considerations_for_implementing_sr-iov/index. [Online; accessed: 23-October-2025].

[9] Inc. Advanced Micro Devices. *Alveo U55C Data Center Accelerator Cards Data Sheet (DS978)*. Version 1.3. 2023. URL: https://docs.amd.com/viewer/book-attachment/jBy7Lf0ukySp4TFAfKhJJQ/aj6Aw4v67dCW~TvHEOe5UQ-jBy7Lf0ukySp4TFAfKhJJQ. [Online; accessed: 23-October-2025].

[10] ARM Limited. *AMBA® AXI-Stream Protocol Specification*. 2021. URL: https://developer.arm.com/documentation/ihi0051/latest/. [Online; accessed: 23-October-2025].

[11] Inc. Advanced Micro Devices. *UltraScale+ Devices Integrated Block for PCI Express v1.3. Product Guide*. PG213 (v1.3). 2025. URL: https://docs.amd.com/viewer/book-attachment/tCJDwogjyJ9_CE2Q_px23A/V1qgleVOyfTzj42ioh4SWw-tCJDwogjyJ9_CE2Q_px23A. [Online; accessed: 23-October-2025].

[12] Inc NVM Express. *NVMe Overview*. URL: https://nvmexpress.org/wp-content/uploads/NVMe_Overview.pdf. [Online; accessed: 23-October-2025].

[13] Vladislav Válek. *DMA Iuventus*. URL: https://github.com/walliv/ndk-fpga/tree/valek-feat-dma_iuventus/comp/dma/dma_iuventus. [Online; accessed: 13-March-2026].

[14] CESNET a.l.e. *Network Development Kit (NDK) for FPGA cards*. URL: https://github.com/CESNET/ndk-fpga. [Online; accessed: 23-October-2025].

[15] CESNET a.l.e. *Linux driver and SW tools for Network Development Kit (NDK)*. URL: https://github.com/CESNET/ndk-sw. [Online; accessed: 4-March-2026].

[16] Ziye Yang et al. "SPDK: A Development Kit to Build High Performance Storage Applications". In: *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2017, pp. 154–161. DOI: 10.1109/CloudCom.2017.14.

[17] The kernel development community. *The Userspace I/O HOWTO*. Version 7.0.0-rc5. URL: https://docs.kernel.org/driver-api/uio-howto.html. [Online; accessed: 24-March-2026].

[18] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers, Third Edition*. O'Reilly Media, Inc., 2005. ISBN: 0-596-00590-3.

[19] Data Plane Development Kit. *7. Linux Drivers*. Version 26.03.0-rc2. 2026. URL: https://doc.dpdk.org/guides/linux_gsg/linux_drivers.html. [Online; accessed: 23-March-2026].

[20] SPDK. *User Space Drivers*. URL: https://spdk.io/doc/userspace.html. [Online; accessed: 13-March-2026].

[21] SPDK. *Direct Memory Access (DMA) From User Space*. URL: https://spdk.io/doc/memory.html. [Online; accessed: 23-March-2026].

[22] NVM Express Inc. *Non-Volatile Memory Express*. Version 1.2b. June 1, 2016. URL: https://nvmexpress.org/wp-content/uploads/NVM_Express_1_2b_20160601-1.pdf. [Online; accessed: 13-March-2026].

[23] CESNET a.l.e. *MI bus specification*. URL: https://cesnet.github.io/ndk-fpga/devel/comp/mi_tools/readme.html. [Online; accessed: 23-October-2025].

[24] CESNET a.l.e. *MFB specification*. URL: https://cesnet.github.io/ndk-fpga/devel/comp/mfb_tools/readme.html. [Online; accessed: 23-October-2025].

[25] Vladislav Válek, Martin Špinler, and Jakub Cabal. *DMA Calypte*. 2026. URL: https://cesnet.github.io/ndk-fpga/devel/comp/dma/dma_calypte/readme.html. [Online; accessed: 23-October-2025].

[26] Jens Axboe. *Efficient IO with io_uring*. Version 0.4. Oct. 15, 2019. URL: https://kernel.dk/io_uring.pdf. [Online; accessed: 19-March-2026].

[27] SPDK. *ublk Target*. URL: https://spdk.io/doc/ublk.html. [Online; accessed: 19-March-2026].

[28] The kernel development community. *Userspace block device driver (ublk driver)*. Version 7.0.0-rc4. URL: https://docs.kernel.org/block/ublk.html. [Online; accessed: 19-March-2026].

[29] Microsoft Inc. *exFAT file system specification*. Version 7. July 10, 2025. URL: https://learn.microsoft.com/en-us/windows/win32/fileio/exfat-specification. [Online; accessed: 13-March-2026].

[30] Arch manual pages. *mkfs.exfat(8)*. URL: https://man.archlinux.org/man/mkfs.exfat.8. [Online; accessed: 19-March-2026].

[31] Linaro Limited. *Devicetree Specification document source files*. Version 0.4. June 28, 2024. URL: https://github.com/devicetree-org/devicetree-specification/releases/tag/v0.4. [Online; accessed: 13-March-2026].

[32] AMD Inc. *UltraScale Architecture Libraries User Guide (UG974). SYSMONE4*. Version 2025.1. May 29, 2025. URL: https://docs.amd.com/r/2025.1-English/ug974-vivado-ultrascale-libraries/SYSMONE4. [Online; accessed: 13-March-2026].

[33] AMD Inc. *UltraScale Architecture Libraries User Guide (UG974). ICAPE3*. Version 2025.1. May 29, 2025. URL: https://docs.amd.com/r/2025.1-English/ug974-vivado-ultrascale-libraries/ICAPE3. [Online; accessed: 13-March-2026].

[34] AMD Inc. *Integrated Logic Analyzer (ILA)*. 2026. URL: https://www.amd.com/de/products/adaptive-socs-and-fpgas/intellectual-property/ila.html. [Online; accessed: 13-March-2026].

[35] Allyn Melaventano. *Intel's Optane DC Persistent Memory DIMMs Push Latency Closer to DRAM*. URL: https://pcper.com/2018/12/intels-optane-dc-persistent-memory-dimms-push-latency-closer-to-dram/. [Online; accessed: 19-March-2026].

[36] Nick Gasson. *nvc*. Version 1.18.2. Nov. 19, 2025. URL: https://github.com/nickg/nvc/tree/r1.18.2. [Online; accessed: 19-March-2026].

[37] cocotb. *cocotb | Python verification framework*. 2026. URL: https://www.cocotb.org/. [Online; accessed: 19-March-2026].

[38] cocotb. *cocotb-bus | Pre-packaged testbenching tools and reusable bus interfaces for cocotb*. Jan. 30, 2026. URL: https://github.com/cocotb/cocotb-bus. [Online; accessed: 19-March-2026].

[39] CESNET a.l.e. *ndk-fpga*. Version valek-feat-dma_iuventus. Mar. 19, 2025. URL: https://github.com/walliv/ndk-fpga/tree/valek-feat-dma_iuventus. [Online; accessed: 19-March-2026].

[40] AMD Inc. *Vivado Design Suite User Guide: Programming and Debugging (UG908)*. Version 2025.1. May 29, 2025. URL: https://docs.amd.com/r/2025.1-English/ug908-vivado-programming-debugging/Programming-the-Device. [Online; accessed: 20-March-2026].

[41] AMD Inc. *Vivado Design Suite User Guide: Programming and Debugging (UG908)*. Version 2025.1. May 29, 2025. URL: https://docs.amd.com/r/2025.1-English/ug908-vivado-programming-debugging/Connecting-to-a-Remote-hw_server-Running-on-a-Lab-Machine. [Online; accessed: 20-March-2026].

[42] The kernel development community. *HugeTLB Pages*. Version 7.0.0-rc4. 2026. URL: https://docs.kernel.org/admin-guide/mm/hugetlbpage.html. [Online; accessed: 20-March-2026].

[43] Data Plane Development Kit. *2. System Requirements*. Version 26.03.0-rc2. 2026. URL: https://doc.dpdk.org/guides/linux_gsg/sys_reqs.html. [Online; accessed: 20-March-2026].

[44] spdk. *spdk*. Version feat-fpga_zero_copy. 2026. URL: https://github.com/walliv/spdk/tree/feat-fpga_zero_copy. [Online; accessed: 20-March-2026].

Symbols and abbreviations

AT	Address Translation
ACS	Access Control Services
ACQ	Admin Completion Queue Base Address
AQA	Admin Queue Attributes
ASQ	Admin Submission Queue Base Address
BAR	Base Address Register
BDF	Bus, Device, Function
DNR	Do Not Retry
DPTR	Data Pointer
CAP	Controller Capabilities
CC	Completer Completion
CEM	Card Electromechanical
CID	Command Identifier
Cpl	Completion
CQ	Completer Request
CQE	Completion Queue Entry
CQHDBL	Completion Queue Head Doorbell
CQTDBL	Completion Queue Tail Doorbell
C2H	Card-to-Host
C/S	Configuration and Status
DLLP	Data-link Layer Packet
DMA	Direct Memory Access
DPDK	Data Plane Development Kit
D2D	Device-to-Device

D2H Device-to-Host

EP Endpoint

FAT File Allocation Table

exFAT extensible File Allocation Table

FBE First Byte Enable

FPGA Field-programmable Gate Array

H2C Host-to-Card

H2D Host-to-Device

ICAP Internal Configuration Access Point

ILA Integrated Logic Analyzer

IOMMU I/O Memory Management Unit

IOVA I/O Virtual Address

LBA Logical Block Address

LBE Last Byte Enable

MDTS Maximum Data Transfer Size

MFB Multi-frame Bus

MI Memory Interface

MMIO Memory-mapped I/O

MPS Maximum Payload Size

MRd Memory Read Request

MRRS Maximum Read Request Size

MWr Memory Write Request

NIC Network Interface Card

NLB Number of Logical Blocks

NSID Namespace Identifier

NVM	Non-volatile Media
NVMe	Non-volatile Memory Express
NVMe-oF	NVMe over Fabric
OPC	Opcode
PCI	Peripheral Component Interconnect
PCIe	Peripheral Component Interconnect Express
PRP	Physical Resource Page
PRP1	PRP Entry 1
PRP2	PRP Entry 2
P2P	Peer-to-peer
QID	Queue Identifier
RC	Root Complex
RC	Requester Completion
RQ	Requester Request
SC	Status Code
SCT	Status Code Type
SLBA	Starting LBA
SPDK	Storage Performance Development Kit
SQE	Submission Queue Entry
SQID	SQ Identifier
SQHDBL	Submission Queue Head Doorbell
SQTDBL	Submission Queue Tail Doorbell
SSD	Solid-state Drive
TC	Transaction Class
TLP	Transaction Layer Packet
ublk	Userspace block device

List of appendices

A Installation and configuration guide

The installation process was conducted on an Arch-based OS, and installation of some prerequisites as well as naming of your distribution's packages may vary. It is recommended to use a Python virtual environment or `conda`/`mamba` to install Python modules so these do not clash with system-wide installation. Make sure that you have sudo privileges on the PC the FPGA card is installed to. The main prerequisites are:

- AMD Vivado 2025.1
- gcc (tested with version 15.1.1)
- dtc (DeviceTree compiler, tested on v1.7.2-1-g9af601c)
- (optionally) mamba (workspace tool to manage Python environments)
- (optionally) Packages for simulation
  - nvc (VHDL simulator, strictly v1.18.2)[36]
  - cocotb (Python simulation framework, strictly v2.0.1)[37]
  - cocotb-bus (cocotb addon with generic bus models)[38]

The installation and configuration are described from the perspective of a single computer and consist of multiple steps that should be performed in the following order:

1. building and installing the NDK-SW framework with its kernel driver, C library and Python modules
2. installing Python modules from the NDK-FPGA framework
3. building the FPGA design from the NDK-FPGA framework
4. connecting an FPGA card and programming it with the obtained bitstream
5. checking that a card has been correctly initialized
6. building the SPDK framework
7. unbinding of devices
8. attaching the ublk driver to the kernel

Finally, we introduce how to proceed with new FPGA designs with minimal steps for reconfiguration.

A.1 Installing ndk-sw

The *ndk-sw* repository[15] should be cloned and entered. The documentation provides a list of steps for installing the `nfb-framework` package. The Python modules should be installed inside a virtual environment. The user can use a starter YAML file located in the `apps/iuventus/sw/` directory in the NDK-FPGA repository. RHEL-based systems allow to download and install prebuilt packages without the need to perform the following steps:

Listing A.1: Installing the NDK-SW package and Python modules

```
# (optional) Deactivate mamba/conda environment (sometimes, it
    needs to be called multiple times)
$ mamba deactivate

# Clone the repository
$ git clone https://github.com/CESNET/ndk-sw
$ cd ndk-sw/

# Installing prerequisites required by ndk-sw
$ sudo ./build.sh --bootstrap

cd package/
# Build the package (specific command for Arch-based OS)
$ makepkg -s

# The command will build all of the packages and store them in the
    current directory as *.pkg.tar.zst archives

# Install the created nfb-framework package (the versions can
    differ based on the current release)
$ sudo pacman -U nfb-framework-6.30.1-1-x86_64.pkg.tar.zst

# Install Python API to the virtual environment
$ mamba activate my-env
$ cd ../pynfb
$ python -m pip install .

# The system should be restarted now. Otherwise the installed
    tools would not work on the initialized hardware.
$ sudo reboot
```

After the installation of the packages, proceed to FPGA bitstream generation.

A.2 Building FPGA design

The NDK-FPGA is contained within its GitHub repository[39] where the branch `valek-feat-dma_iuventus` needs to be switched into. The design is built using the following steps:

Listing A.2: Building the NDK-FPGA design

```
# Clone the repository and switch to the required branch
$ git clone git@github.com:walliv/ndk-fpga.git
$ cd ndk-fpga
$ git switch valek-feat-dma_iuventus
```

```
# Descend in the build directory of the Iuventus app
$ cd apps/iuventus/build/alveo-u55c
# Start the build of the design
$ make

# The build is done using Vivado which can take several minutes.
   After this finishes, there are several files to notice:
$ ls -l
```

The synthesis reports are available in `*_synth.drc`, `*_synth.tim` and `*_synth.util` files whereas the reports from implementation in `*_par.drc`, `*_par.tim` and `*_par.util` files. The generated DeviceTree can be seen in `*.netcope_tmp/DeviceTree.dts` file. There is the Vivado-specific bitstream as the `*.bit` file and its NDK-FPGA-specific option in the `*.nfw` file. The `*.nfw` file is basically an archive containing the `*.bit` bitstream and the DeviceTree file used by the NDK-SW programming tool for protection to check that the bitstream gets programmed into a correct hardware.

The NDK-FPGA framework contains necessary Python sources to interact with different hardware components using the Python API installed as a part of NDK-SW. It is recommended to install these modules inside a Python or Conda/Mamba virtual environment.

Listing A.3: Installing NDK-FPGA Python modules

```
$ mamba activate my-env

# Go to the NDK-FPGA and source shell environment file
$ source env.sh

# Install utilities to operate with physical hardware
$ cd python/ofm
$ python -m pip install .

# Install utilities for general models (e.g. register list)
$ cd ../cocotbext
$ python -m pip install .
```

The Python modules as well as the compiled bitstream are now ready. We can now proceed to the programming of the hardware.

A.3 Programming the device

The first setup needs to be done in the UEFI of the computer:

- enabling 64-bit addressing for PCIe devices (sometimes called "Above 4G encoding")

- disabling the IOMMU (when possible, otherwise described further).
- disabling ACS
- enabling PCIe connectors to run at Gen4 data rates using at least 8 lanes (can be left on auto)

The NDK-SW framework provides the easy way to program FPGA accelerator cards with the `nfb-boot` tool. This accepts both, the `*.bit` and the `*.nfw` files as bitstreams, where the second variant is preferred. However, a card this tool is about to be applied to needs to run a previous NDK-FPGA design which contains necessary components to load the new bitstream into the configuration flash. The tool is used in the following way:

Listing A.4: Programming using the nfb-boot tool

```
# Programs the specified bitstream into configuration slot 0
$ nfb-boot -f0 alveo-u55c-iuventus-pcie1xGen4x8.nfw

# The programming takes some time and its status is shown by the
   progress bar. Do not interrupt this process unless you want to
   corrupt the loaded design and the boot process

# For the design used in this thesis that contains two PCIe
   Functions, the reboot of the system is also necessary
$ sudo reboot
```

In case an acceleration card does not currently run an NDK-FPGA design, the standard way of programming through *JTAG* needs to be used. Most FPGA accelerators contain the *Micro-USB* connector that operates this programming interface and the *hardware server* provided within the Vivado environment connects to it. The programming through this is described in the manual [40]. If the Vivado is running on the same machine as the FPGA accelerator, the hardware server is run automatically when opening the *Hardware Manager* inside Vivado GUI and clicking on the `Auto Connect`. This will locate all locally attached JTAG-compliant devices that can be programmed.

In case the FPGA acceleration card is located on a remote machine rather than the one Vivado is running on, a slightly different approach needs to be used by starting the hardware server remotely as described in [41]. The remote server needs at least the *Vivado Lab* edition installed. The Hardware Manager needs to be opened on the local machine but the `Auto Connect` feature will not work this time and the remote server needs to be explicitly specified as a target through the `Open New Target...` option. Notice that the remote and local installations of the Vivado environment need to be of the same version. If your card is already running a design, connecting to the `hw_server` is necessary if you are using ILA cores in your design which is also the case of this thesis to observe received data that have

been read from the NVMe device. To observe signals on the ILA, two files have to be supplied to the opened target where the first is the configuration bitstream (`*.bit`) and the second is the file containing probe descriptions (`*.ltx`).

Listing A.5: Running hardware server on the remote machine

```
# This will run the hardware server in the background listening on
    port 3121
$ /opt/Xilinx/2025.1/Vivado_Lab/bin/hw_server -p0 &

****** Xilinx hw_server v2025.1
  **** Build date : May  6 2025 at 15:14:51
    ** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
    ** Copyright 2022-2025 Advanced Micro Devices, Inc. All Rights
       Reserved.

INFO: hw_server application started
INFO: Use Ctrl-C to exit hw_server application

INFO: To connect to this hw_server instance use url:
   TCP:hostname:3121
```

Contrary to the previous listing, the address of the `hw_server` instance is actually the whole domain name within the internet network followed by the port number: `hostname@domain:3121`, like `my_computer@institute.uni-nowhere.de:3121`. After resetting the test machine, the programmed device can now be detected by the NDK-SW tools.

Listing A.6: Showing running devices using NDK-SW tools

```
# Primary tool that shows all FPGA accelerators with an NDK-FPGA
    design running
$ nfb-info -l
ID  Base path   PCI address   Card name         Serial number
    Firmware info - project
 0  /dev/nfb0   0000:41:00.0  ALVEO_U55C        0
    IUVENTUS_TEST  1xGen4x8  0.12.0
 1  /dev/nfb1   0000:01:00.0  ALVEO_U55C        0
    IUVENTUS_TEST  1xGen4x8  0.12.0
 2  /dev/nfb2   0000:02:00.0  ALVEO_U200        0
    NDK_MINIMAL  100G2  0.11.0

# The devices are implicitly numbered by their ID (default is 0).
    Without flags, the tool shows the device with index 0.
$ nfb-info
---------------------------------------- Board info ----
Board name                 : COMBO-GENERIC
Serial number              : 0
```

```
Network interfaces          : 0
------------------------------------ Firmware info ----
Card name                   : ALVEO_U55C
Project name                : IUVENTUS_TEST
Project variant             : 1xGen4x8
Project version             : 0.12.0
Built at                    : 2026-03-18 19:27:06
Build author                : vladislav.valek@email.cz
Build revision              : c08caf5ef
RX queues                   : 0
TX queues                   : 0
ETH channels                : 0
-------------------------------------- System info ----
PCIe Endpoint 0:
 * PCI slot                 : 0000:41:00.0
 * PCI link speed           : 16 GT/s
 * PCI link width           : x8
 * NUMA node                : -1
 * MI BAR 0 size            : 64 MiB

# The important values are in the PCIe Endpoint 0 section that
   show that the design that is running the DMA Iuventus (marked
   as IUVENTUS_TEST project name) has a correct configuration of
   Gen4x8.
# Other devices can be checked using the -d flag with the ID
$ nfb-info -d2
---------------------------------------- Board info ----
Board name                  : COMBO-GENERIC
Serial number               : 0
Network interfaces          : 0
------------------------------------ Firmware info ----
Card name                   : ALVEO_U200
Project name                : NDK_MINIMAL
Project variant             : 100G2
Project version             : 0.11.0
Built at                    : 2025-10-21 11:17:12
Build author                : vladislav.valek@email.cz
Build revision              : abed9a4ae
RX queues                   : 16 (only 0 available)
TX queues                   : 16 (only 0 available)
ETH channels                : 0
-------------------------------------- System info ----
PCIe Endpoint 0:
 * PCI slot                 : 0000:02:00.0
 * PCI link speed           : 8 GT/s
 * PCI link width           : x16
 * NUMA node                : -1
```

```
 * MI BAR 0 size            : 64 MiB
 * MI BAR 2 size            : 16 MiB
```

A.4 Building the SPDK application

If the IOMMU cannot be disabled by the UEFI settings, it can be disabled in another way by instructing the kernel to not use it at all which is done by kernel parameters. Another kernel parameter setting for this thesis encompasses the allocation of *hugepages*. These allow to use pages of bigger size than the standard 4 KiB, like 8 KiB, 1 MiB and even 1 GiB[42]. The SPDK framework runs its core on the existing *Data Plane Development Kit* (DPDK) software stack used for fast userspace packet processing using specialized NICs. This stack recommends to allocate multiple 1 GiB hugepages if the architecture supports such size[43].

Listing A.7: Setting kernel command line parameters

```
# Edit the GRUB_CMDLINE_LINUX variable, for example (the "quiet
   loglevel=3" are just examples that can be already present from
   the previous edit):
# GRUB_CMDLINE_LINUX="quiet loglevel=3 amd_iommu=off
   default_hugepagesz=1GB hugepagesz=1G hugepages=10"
$ sudo vim /etc/default/grub

# Afterwards, the GRUB configuration needs to be refreshed (this
   is a permanent setting)
$ sudo grub-mkconfig -o /boot/grub/grub.cfg
# OR some OSes use a different command
$ sudo update-grub

# And reboot the system
$ sudo reboot

# The currently applied parameters can be checked by reading
   /proc/cmdline
$ cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-linux-lts
   root=UUID=38396817-558e-4cd4-a890-a94c8ec57c60 rw
   default_hugepagesz=1GB hugepagesz=1G hugepages=10 quiet
   loglevel=3 audit=0 nvme_load=yes amd_iommu=off

# The allocated hugepages can be checked reading /proc/meminfo
$ cat /proc/meminfo
... some output omitted ...
HugePages_Total:      10
HugePages_Free:       10
```

```
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:    1048576 kB
Hugetlb:        10485760 kB
... some output omitted ...

# To check that the IOMMU is disabled, the following directory is
    empty
$ ls -l /sys/kernel/iommu_groups/
total 0
```

After verifying the kernel configuration, load the required userspace drivers:

Listing A.8: Loading userspace drivers

```
$ sudo modprobe uio-pci-generic
$ sudo modprobe ublk_drv
```

After the initial configuration, the FPGA accelerator's function 1 and the NVMe drive can now be unbound. The concrete BDF identifiers can vary.

Listing A.9: Unbinding PCIe devices

```
# Figure out the Vendor ID and Device ID numbers of the devices
$ lspci -nn
        ... output omitted ...
41:00.0 Memory controller [0580]: Cesnet, z.s.p.o. Device
    [18ec:c000]
41:00.1 Memory controller [0580]: Cesnet, z.s.p.o. Device
    [18ec:c020]
        ... output omitted ...
61:00.0 Non-Volatile memory controller [0108]: SK hynix PC611 NVMe
    Solid State Drive [1c5c:1639]
        ... output omitted ...

# The output shows the two functions of the FPGA accelerator,
    41:00.0 and 41:00.1 and a one-function NVMe drive 61:00.0. The
    VID and DID numbers are written in square brackets on the end
    of every entry.

# Write VID and DID to the uio_pci_generic driver
$ echo "18ec␣c020" | sudo tee
    /sys/bus/pci/drivers/uio_pci_generic/new_id
$ echo "1c5c␣1639" | sudo tee
    /sys/bus/pci/drivers/uio_pci_generic/new_id

# Bind the FPGA accelerator device to the uio_pci_generic driver
$ echo 0000:41:00.1 | sudo tee
    /sys/bus/pci/drivers/uio_pci_generic/bind
```

```
# Enable access with memory transactions, Bus Master attribute and
    disable pin-based interrupts
$ sudo setpci -s41:00.1 COMMAND=0x406

# The NVMe drive has to be first unbound from its native driver
    (nvme)...
$ echo '0000:61:00.0' | sudo tee
    /sys/bus/pci/devices/0000\:61\:00.0/driver/unbind
# ... and rebound to the uio_pci_generic
$ echo "0000:61:00.0" | sudo tee
    /sys/bus/pci/drivers/uio_pci_generic/bind
# The PCIe parameters are the same as in the case of the FPGA
    accelerator
$ sudo setpci -s61:00.0 COMMAND=0x406
```

The hardware is now ready to run the SPDK framework that can be cloned from the fork of its GitHub repository[44] and switching to branch `feat-fpga_zero_copy`

Listing A.10: Building the SPDK application

```
$ git clone git@github.com:walliv/spdk.git --recursive
$ git switch feat-fpga_zero_copy

# Install dependencies with the io_uring support
$ sudo scripts/pkgdep.sh -u

# Configure framework with the ublk support and debug enabled and
    run build
$ ./configure --with-ublk --enable-debug
$ make
```

The build of the SPDK application will take some time ending with all executables stored in the `build/` directory. There are several tools that can be tried to verify that an NVMe is connected.

Listing A.11: Testing NVMe drive using native SPDK tools

```
$ cd build/bin
# Calling the Identify command to read important information about
    state and allowed parameters of the drive (the 0000:61:00.0 is
    the drive that we previously unbound)
$ sudo ./spdk_nvme_identify -r "trtype:PCIe␣traddr:0000:61:00.0"

# Measuring performance of the NVMe drive with different
    parameters:
# Testing the drive with the queue of size 1 by reading 512 bytes
    for 10 seconds
$ sudo ./spdk_nvme_perf -r "trtype:PCIe␣traddr:0000:61:00.0" -q1
    -o512 -w read -t 10
```

```
Initializing NVMe Controllers
Attached to NVMe Controller at 0000:61:00.0 [1c5c:1639]
Associating PCIE (0000:61:00.0) NSID 1 with lcore 0
Initialization complete. Launching workers.
========================================================
              Latency(us)
Device Information                      :       IOPS      MiB/s
   Average         min         max
PCIE (0000:61:00.0) NSID 1 from core  0:   20948.50      10.23
        47.73      45.78    2924.73
========================================================
Total                                   :   20948.50      10.23
        47.73      45.78    2924.73
```

The installation is now ready and the examples in Section 6.3 can be tried out.

B DMA Iuventus register map

The complete list of registers that are initialized in the *Software manager* component is listed in table B.1.

- Each register holds 32 bits, thus, when more bits are required, more registers are used to hold bigger values (denoted with either `L` and `H` in the Name column when only two registers are required or with indexes when more than 2 registers are used).
- The registers are of two types, namely the standard ones holding status information or used for configuration (marked as `REG` in the table); and counters that hold sampled values of statistical counters (marked as `CNTR` in the Type column).
- The `SW perm.` column shows which read/write permissions to the specific register does the configuration software have.
- The `HW perm.` column shows which read/write permissions to the specific register does the DMA Iuventus module have.
- Some registers have a software Read permission marked as *Strobe* which marks registers that need to be sampled before the current value can be read from them. This is done by writing bit 1 in the *Control* register. This sampling is necessary because signals that span over multiple registers can change their value between two subsequent register reads and thus return inaccurate information.

Table B.1: The complete list of registers of the DMA Iuventus module

Index	**Addr (dec)**	**Addr (hex)**	**Type**	**Name**	**SW perm.**	**HW perm.**	**Width [bits]**	**Notes**
0	0	0	REG	Control	RW	R	5	0: Design enable 1: Sample counters 2: Clear error mask 3: Counters reset 4: Enable update repeat
1	4	4	REG	Status	R	W	3	0: Design ready 1: RST Done 2: Tag Initialization Done
2	8	8	REG	SQTDBL	R	W	16	
3	12	C	REG	SQHDBL	R	W	16	
4	16	10	REG	CQHDBL	R	W	16	
5	20	14	REG	DBL Mask	RW	R	16	Mask for all DBL registers, determines size of queues
6	24	18	REG	SQTDBL Base Addr L	RW	R	32	Base address in NVMe BAR containing SQTDBL register for the FPGA queue pair
7	28	1C	REG	SQTDBL Base Addr H	RW	R	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
8	32	20	REG	CQHDBL Base Addr L	RW	R	32	Base address in NVMe BAR containing CQHDBL register for the FPGA queue pair
9	36	24	REG	CQHDBL Base Addr H	RW	R	32	
10	40	28	REG	RD Buff Base Addr L	RW	R	32	FPGA BAR Address containing the Read Buffer (to properly assign PRP Entries in the SQEs)
11	44	2C	REG	RD Buff Base Addr H	RW	R	32	
12	48	30	REG	RD Buff PRP List Base L	RW	R	32	The physical address to the host memory of a generated PRP list for read buffer
13	52	34	REG	RD Buff PRP List Base H	RW	R	32	
14	56	38	REG	WR Buff Base Addr L	RW	R	32	FPGA BAR Address containing the Write Buffer (to properly assign PRP Entries in the SQEs)
15	60	3C	REG	WR Buff Base Addr H	RW	R	32	
16	64	40	REG	WR Buff PRP List Base L	RW	R	32	The physical address to the host memory of a generated PRP list for write buffer
17	68	44	REG	WR Buff PRP List Base H	RW	R	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
18	72	48	REG	Last CQ Entry 0	R (Strobe)	W	32	Last CQ Entry read from the Completion Queue
19	76	4C	REG	Last CQ Entry 1	R (Strobe)	W	32	
20	80	50	REG	Last CQ Entry 2	R (Strobe)	W	32	
21	84	54	REG	Last CQ Entry 3	R (Strobe)	W	32	
22	88	58	CNTR	SQEs Dispatched L	R (Strobe)	W	32	The amount of dispatched SQEs
23	92	5C	CNTR	SQEs Dispatched H	R (Strobe)	W	32	
24	96	60	CNTR	CQEs Processed L	R (Strobe)	W	32	The amount of processed CQEs
25	100	64	CNTR	CQEs Processed H	R (Strobe)	W	32	
26	104	68	CNTR	PCIe Reads L	R (Strobe)	W	32	The amount of arrived PCIe reads
27	108	6C	CNTR	PCIe Reads H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
28	112	70	CNTR	PCIe Read Bytes L	R (Strobe)	W	32	The amount of bytes requested in arrived PCIe reads
29	116	74	CNTR	PCIe Read Bytes H	R (Strobe)	W	32	
30	120	78	CNTR	PCIe Writes L	R (Strobe)	W	32	The amount of arrived PCIe writes
31	124	7C	CNTR	PCIe Writes H	R (Strobe)	W	32	
32	128	80	CNTR	PCIe Write Bytes L	R (Strobe)	W	32	The amount of bytes received in arrived PCIe writes
33	132	84	CNTR	PCIe Write Bytes H	R (Strobe)	W	32	
34	136	88	CNTR	SQ PCIe Reads L	R (Strobe)	W	32	The amount of PCIe reads from the SQ
35	140	8C	CNTR	SQ PCIe Reads H	R (Strobe)	W	32	
36	144	90	CNTR	SQ PCIe Read Bytes L	R (Strobe)	W	32	The amount of bytes requested in PCIe reads from the SQ
37	148	94	CNTR	SQ PCIe Read Bytes H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
38	152	98	REG	LBA Num Mask	RW	R	16	Determines the maximum amount of LBAs that can be read/written by one command
39	156	9C	CNTR	Successful Completions L	R (Strobe)	W	32	The amount of successful completions
40	160	A0	CNTR	Successful Completions H	R (Strobe)	W	32	
41	164	A4	CNTR	Unsuccessful Completions L	R (Strobe)	W	32	The amount of unsuccessful completions
42	168	A8	CNTR	Unsuccessful Completions H	R (Strobe)	W	32	
43	172	AC	REG	Completion Error Mask L	R	W	32	Bit mask of captured completion errors based on the error set in the specification v1.2b
44	176	B0	REG	Completion Error Mask H	R	W	32	
45	180	B4	CNTR	RD Buff PCIe Reads L	R (Strobe)	W	32	The amount of PCIe reads from the Read buffer
46	184	B8	CNTR	RD Buff PCIe Reads H	R (Strobe)	W	32	
47	188	BC	CNTR	RD Buff PCIe Read Bytes L	R (Strobe)	W	32	The amount of bytes requested in PCIe reads from the Read Buffer

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
48	192	C0	CNTR	RD Buff PCIe Read Bytes H	R (Strobe)	W	32	
49	196	C4	CNTR	WR Buff PCIe Writes L	R (Strobe)	W	32	The amount of PCIe writes to the Write Buffer
50	200	C8	CNTR	WR Buff PCIe Writes H	R (Strobe)	W	32	
51	204	CC	CNTR	WR Buff PCIe Wr Bytes L	R (Strobe)	W	32	The amount of bytes received in PCIe writes to the Write Buffer
52	208	D0	CNTR	WR Buff PCIe Wr Bytes H	R (Strobe)	W	32	
53	212	D4	REG	LBA Space Size L	RW	R	32	Size of the NVMe drive in LBAs
54	216	D8	REG	LBA Space Size H	RW	R	32	
55	220	DC	CNTR	CQ PCIe Writes L	R (Strobe)	W	32	The amount of PCIe writes to the Completion Queue
56	224	E0	CNTR	CQ PCIe Writes H	R (Strobe)	W	32	
57	228	E4	CNTR	CQ PCIe Write Bytes L	R (Strobe)	W	32	The amount of bytes received in PCIe writes to the Completion Queue
58	232	E8	CNTR	CQ PCIe Write Bytes H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
59	236	EC	CNTR	CQHDBL regular upds L	R (Strobe)	W	32	The amount of regular CQHDBL updates (i.e. updates dispatched as a consequence of a freshly processed CQE)
60	240	F0	CNTR	CQHDBL regular upds H	R (Strobe)	W	32	
61	244	F4	CNTR	CQHDBL repeated upds L	R (Strobe)	W	32	The amount of repeated CQHDBL updates (i.e. updates repeated in order to ensure the most current value in the NVMe registers)
62	248	F8	CNTR	CQHDBL repeated upds H	R (Strobe)	W	32	
63	252	FC	CNTR	SQTDBL regular upds L	R (Strobe)	W	32	The amount of regular SQTDBL updates (i.e. updates dispatched as a consequence of a freshly dispatched SQE)
64	256	100	CNTR	SQTDBL regular upds H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
65	260	104	CNTR	SQTDBL repeated upds L	R (Strobe)	W	32	The amount of repeated SQTDBL updates (i.e. updates repeated in order to ensure the most current value in the NVMe registers)
66	264	108	CNTR	SQTDBL repeated upds H	R (Strobe)	W	32	
67	268	10C	REG	Metadata pointer L	RW	R	32	(unused) Pointer to metadata region (for metadata field in SQE, assigned to some dummy address in the host memory)
68	272	110	REG	Metadata pointer H	RW	R	32	(unused)
69	276	114	CNTR	NVME RD Cmd Bytes L	R (Strobe)	W	32	The amount of bytes requested in NVMe read commands
70	280	118	CNTR	NVME RD Cmd Bytes H	R (Strobe)	W	32	
71	284	11C	CNTR	NVME WR Cmd Bytes L	R (Strobe)	W	32	The amount of bytes requested in NVMe write commands
72	288	120	CNTR	NVME WR Cmd Bytes H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
73	292	124	CNTR	WR Buff Usr Reads L	R (Strobe)	W	32	The amount of read packets from the write buffer
74	296	128	CNTR	WR Buff Usr Reads H	R (Strobe)	W	32	
75	300	12C	CNTR	WR Buff Usr Read Bytes L	R (Strobe)	W	32	The amount of read bytes from the write buffer
76	304	130	CNTR	WR Buff Usr Read Bytes H	R (Strobe)	W	32	
77	308	134	CNTR	RD Buff Disp Reads L	R (Strobe)	W	32	The amount of read packets from the read buffer
78	312	138	CNTR	RD Buff Disp Reads H	R (Strobe)	W	32	
79	316	13C	CNTR	RD Buff Disp Read Bytes L	R (Strobe)	W	32	The amount of read bytes from the read buffer
80	320	140	CNTR	RD Buff Disp Read Bytes H	R (Strobe)	W	32	
81	324	144	CNTR	SQ Disp Reads L	R (Strobe)	W	32	The amount of read packets from the submission queue
82	328	148	CNTR	SQ Disp Reads H	R (Strobe)	W	32	

Continued on next page

Index	Addr (dec)	Addr (hex)	Type	Name	SW perm.	HW perm.	Width [bits]	Notes
83	332	14C	CNTR	SQ Disp Read Bytes L	R (Strobe)	W	32	The amount of read bytes from the submission queue
84	336	150	CNTR	SQ Disp Read Bytes H	R (Strobe)	W	32	
85	340	154	REG	Tag FIFO Status	R	W	12	The amount of free tags in the FIFO
86	344	158	CNTR	NVME FLUSH Cmds L	R (Strobe)	W	32	The amount of dispatched FLUSH NVMe commands
87	348	15C	CNTR	NVME FLUSH Cmds H	R (Strobe)	W	32	